3 주차 실습 교재

데이터 수집 (구글시트 · Colab) → 저작권 확인 → AI 정제

이 교재 한 권으로 이번 주 실습을 처음부터 끝까지 따라 합니다. 설명을 읽고, 직접 입력해 보고, 칸에 결과를 기록하세요.

구분	내용
Part A	구글시트 IMPORTXML 로 헤드라인 · 기사링크 · 기사원문 수집 (6 열 양식)
Part B	Colab(파이썬)으로 API 키 없이 수집 — RSS · 웹페이지 표 · 목록 페이지, CSV 저장
Part C	Colab 에서 박스오피스 API 호출 코드를 실행해 데이터 수집 · 저장
Part D	구글시트 Apps Script 로 박스오피스를 매일 자동 수집 (시간 트리거)
Part E	저작권 유형 확인 → 2 주차 도메인 챗봇으로 정제 → 사람의 검수
준비물	구글 계정 · 크롬 브라우저 · 2 주차에 만든 도메인 챗봇(Gemini Gems 또는 Claude Projects)

이번 주 실습 흐름

단계	하는 일	도구	남기는 기록
② 수집	웹페이지·API 에서 자료를 표로 모은다	구글시트 · Colab · Apps Script	출처 URL · 수집일시
확인	출처의 저작권·이용 조건을 확인한다	사이트 하단 · 공공누리 표시	저작권 유형 체크
③ 정제	수집한 자료를 도메인 챗봇으로 콘텐츠로 다듬는다	Gemini Gems · Claude Projects	정제 프롬프트 · 결과
검수	원문과 대조해 수치·사실을 확인한다	사람의 눈	대조 결과

먼저 알아둘 3 원칙 — 합법적 AI 데이터 활용

수집한 자료를 AI 에 넣기 전에, 이 자료를 AI 로 다뤄도 되는지 먼저 가립니다.

가능 — AI 로 자유롭게 정제	제한 — 사람이 읽고 내 문장으로
공공데이터포털 통계·수치, 정부·지자체 보도자료 원문	언론사 기사 본문 (AI 에 통째로 넣지 않기)
증시 시세, 경기 스코어 등 사실 수치	유료 구독 콘텐츠
위키백과, 공유마당(CCL) 콘텐츠	상업용 이미지 · 유료 음원
내가 직접 취재·인터뷰한 내용	AI 학습·활용 금지가 명시된 저작물, 공공누리 ‘변경금지’ 유형

1. ① **합법적 수집 구분** — 가져오기 전에 AI 로 다뤄도 되는 출처인지 판단한다.
2. ② **AI 기반 정제** — ①을 통과한 자료는 AI 로 적극 정제·재구성한다.
3. ③ **사람의 검수** — AI 결과는 반드시 사람이 사실관계와 표현을 확인한 뒤 쓴다.

> **왜 이렇게 엄격한가**
> 연합뉴스 같은 통신사는 뉴스를 유료로 공급하는 것이 핵심 수익입니다. 누구나 AI 로 기사를 무단 요약·재가공하면 정당하게 비용을 내는 언론사와의 형평이 깨집니다. 이 원칙은 AI 활용을 막으려는 것이 아니라 콘텐츠를 만든 사람을 보호하려는 장치입니다.

Part A. 구글시트로 수집하기 (IMPORTXML)

수집할 소재 고르기: 소재는 3 주차 폴더의 「3 주차_뉴스브랜드트렌드_데이터출처목록」에서 고릅니다. 뉴스, 사실 데이터, 브랜드, 트렌드 약 50 곳의 주소와 API 지원 여부가 정리되어 있습니다. 그 목록에서 정한 사이트를 C 열에 넣습니다.

A-1. 최종 양식 — 6 열 구조

A·B·C 열은 내가 직접 입력하고, D·E·F 열은 수식이 자동으로 채웁니다.

열	입력	내용
A. 구분	직접	정치 · 경제 · 국제 · 문화 등 대분류
B. 분야	직접	세부 분야 또는 카테고리명
C. URL	직접	수집할 '목록 페이지' 주소
D. 헤드라인	수식	목록 페이지에서 뽑은 기사 제목
E. 기사링크	수식	같은 목록에서 뽑은 개별 기사 주소 (D 열과 같은 행끼리 짝이 맞음)
F. 기사원문	수식	E 열 주소를 읽어 가져온 본문

방식은 2 단계입니다. **① 목록 페이지에서 헤드라인과 링크를 나란히 뽑고, ② 그 링크 열을 참조해 본문을 따로 뽑습니다.**

A-2. 준비 연습 — 함수 한 줄 실행

새 구글시트 A1 에 아래 함수를 그대로 입력합니다.

```
=IMPORTXML("https://ko.wikipedia.org/wiki/대한민국", "//h1")
```

요소	뜻
url	데이터를 가져올 웹페이지 주소 (큰따옴표로 감싸기)
xpath_query	페이지 안에서 가져올 위치를 지정하는 경로. //h1 은 '페이지의 제목 태그'

A1 에 나온 값:

A-3. XPath 얻기 (크롬)

1. 수집할 목록 페이지를 크롬으로 연다.
2. 기사 제목 하나를 우클릭 → **검사**를 누른다.
3. 오른쪽 개발자도구에서 강조된 줄을 우클릭 → **Copy → Copy XPath**.
4. 복사한 경로에서 순번 [1] 등을 지우면 목록 전체를 가져온다. (예: li[1] → li)

따옴표 주의

복사한 XPath 안에 @id="container"처럼 큰따옴표가 있으면, 수식을 감싼 큰따옴표와 겹쳐 오류가 납니다. 속성값 안쪽만 작은따옴표로 바꿉니다: @id='container'. //h1 처럼 단순한 경로는 큰따옴표만으로 충분합니다.

A-4. ① 단계 — 헤드라인 + 기사링크

C2 에 목록 페이지 URL 을 넣고, D2 에 헤드라인 수식, E2 에 링크 수식을 넣습니다. 경로는 같고 E 열은 끝에 /a/@href 가 붙습니다.

```
// D2  헤드라인
=IMPORTXML(C2, "//*[@id='container']/div[2]/div[2]/div[1]/section/div/ul/li/div/div/strong")

// E2  기사링크 (끝에 /a/@href)
=IMPORTXML(C2, "//*[@id='container']/div[2]/div[2]/div[1]/section/div/ul/li/div/div/strong/a/@href")
```

- 경로가 /a/span 까지 가지 않고 /strong 에서 멈춰도 안의 글자를 그대로 가져옵니다.
- li[1]처럼 순번이 있으면 첫 기사 하나만, 순번을 빼면 전체가 여러 행에 자동으로 펼쳐집니다(스필).
- 링크가 '?menukey=...'처럼 일부만 나오면 기본 주소를 앞에 붙입니다: **="https://사이트기본주소"&E2**

내가 쓴 C(목록 URL) · 복사한 XPath:

D 수식 · E 수식:

A-5. ② 단계 — 기사 본문 (F 열)

E 열의 기사링크를 참조해 개별 기사 페이지에서 본문을 가져옵니다. 본문 위치도 같은 방법으로 Copy XPath 합니다.

```
// F5  (E5 의 기사 주소를 읽어 문단 <p>를 모두 이어붙임)
=TEXTJOIN(" ", TRUE, IMPORTXML(E5, "//*[@id='articleWrap']//p"))
```

- **TEXTJOIN** 은 여러 문단을 공백으로 이어 한 셀에 담습니다.
- **//p**(슬래시 두 개)는 '그 안 어디에 있든 p 태그를 모두 찾아라' 는 뜻이라, 구조를 정확히 몰라도 안전합니다.
- 이 수식은 스필이 되지 않습니다. F5 에 넣었다면 아래로 끌어 F6(E6), F7(E7)...로 채웁니다.

F 수식:

A-6. 더 편한 방법 — 셀 참조로 따옴표 문제 없애기

URL 과 XPath 를 각각 셀에 넣어 두고 셀 참조로 쓰면 따옴표 충돌이 사라집니다.

```
// E2 = URL,  F2 = XPath(따옴표 없이 순수 텍스트로 붙여넣기)
=IMPORTXML(E2, F2)
```

- 크롬에서 복사한 XPath 를 변환 없이 그대로 붙여 넣을 수 있습니다.
- 같은 구조의 여러 페이지에는 수식을 아래로 끌기만 하면 됩니다.
- 셀 안에는 순수한 텍스트만 넣습니다. 따옴표까지 문자로 넣으면 오히려 오작동합니다.

A-7. 오류가 났을 때

증상	원인	해결
따옴표 오류	XPath 의 큰따옴표가 수식의 따옴표와 겹침	속성값 안쪽만 작은따옴표로, 또는 셀 참조 방식
#N/A (구조)	그 위치에 일치하는 요소가 없음	//p 처럼 ‘어디든 찾기’ 경로로 넓혀 재시도
#N/A (일시적)	짧은 시간에 호출이 몰려 응답이 꼬임	그 셀만 지우고 다시 입력, 또는 새로고침 (F5). 여러 개가 한꺼번에 실패하면 잠시 후 재시도
‘로드하는 데 시간이 걸림’	시간당 호출 제한 초과	IMPORTXML 개수를 줄이고 잠시 후 재시도
URL 이 한글 제목으로 바뀜	구글시트 스마트 칩이 URL 을 카드로 변환	Ctrl+Shift+V 로 값만 붙여넣기. 이미 바뀌었으면 ‘링크를 텍스트로 변환’
헤드라인이 2~3 개만 나옴	편집자가 고른 소수 기사 영역의 XPath 를 씀	‘전체기사’ 목록 영역에서 다시 Copy XPath
빈 결과	일부 사이트는 본문이 자바스크립트로 나중에 그려짐	IMPORTXML 은 처음 내려받은 HTML 만 읽음 → 파이썬(Part B)도 같은 한계라서 공식 API(Part C) 등 다른 방법을 쓴다

내가 만난 오류와 해결 방법:

A-8. 사이트별 수집·정제 가능 여부 — 직접 시험해 봅니다

"수집이 되는가" 와 "AI 로 정제해도 되는가" 는 **별개**입니다. 두 칸을 따로 확인하세요.

사이트	수집 (IMPORTXML)	AI 정제	비고
연합뉴스	가능 (샘플 확인)	제한	헤드라인과 본문이 수집되지만, 기사 끝에 "AI 학습 및 활용 금지" 가 명시됨 → 본문은 사람이 읽고 재구성
한국소비자원	가능 (샘플 확인)	가능	공공누리 1 유형(출처표시). 기사링크가 '?menukey=...' 상대경로로 나오므로 기본 주소를 앞에 붙임
정책브리핑	직접 시험	가능	정부 정책뉴스, 텍스트는 공공누리 출처표시 조건
식품의약품안전처	직접 시험	가능	리콜·안전 정보 등 보도자료
기상청 날씨누리	직접 시험	가능	공식 예보문, 공공저작물

사이트마다 페이지 구조와 이용 조건이 다릅니다. "직접 시험" 으로 표시한 곳은 이번 실습에서 직접 수집해 보고 결과를 적습니다.

내가 시험한 사이트와 결과 (수집 가능 / 오류, AI 정제 가능 여부):

보조 함수: **IMPORTHTML**(=IMPORTHTML(url, "table" 또는 "list", 순번))은 페이지의 표·목록을 통째로, **IMPORTFEED**(=IMPORTFEED(url))는 RSS 주소로 최신 글 목록을 가져옵니다. XPath 보다 간단합니다.

Part B. Colab(파이썬)으로 API 없이 수집하기

구글시트 함수는 한 번에 몇 개를 가져오기엔 편하지만, 많은 페이지를 반복해서 가져오거나 가져온 뒤 정리·저장하는 과정이 길어집니다. 파이썬(Colab)은 같은 일을 코드로 하고, 표로 정리해 CSV 로 저장하는 것까지 한 흐름으로 합니다. 이 파트는 **인증키 없이** 공개된 통로만 씁니다. 키가 필요한 공식 API 는 Part C 에서 합니다.

B-1. 시트와 파이썬, 무엇이 다른가

구분	구글시트 (Part A)	Colab 파이썬 (Part B)
가져오는 방법	IMPORTXML · IMPORTFEED · IMPORTHTML 수식	requests(페이지 내려받기) · feedparser(RSS) · pandas.read_html(표) · BeautifulSoup(HTML 에서 추출)
위치 지정	크롬 Copy XPath	크롬 Copy selector (CSS 선택자)
많이 가져올 때	호출 제한(“로드하는 데 시간이 걸림”)	반복문 + time.sleep 으로 간격을 조절
가져온 뒤	시트에서 따로 정리	표(pandas)로 정리하고 CSV 저장까지 코드로
같은 한계	처음 내려받은 HTML 만 읽음 (자바스크립트로 나중에 그려지는 내용은 못 가져옴)	requests 도 마찬가지. 이런 사이트는 공식 API(Part C)나 다른 출처를 쓴다

키 없이 쓸 수 있는 통로 3 가지 — ① RSS(최신 글 목록을 주는 공개 주소) ② 웹페이지 안의 표(table) ③ 목록 페이지의 제목·링크. 실습 1~3 에서 하나씩 해 봅니다.

B-2. 코딩 수집에서 지킬 규칙

원칙	내용
인증키 보관	키가 필요한 API 를 쓸 때(Part C) 적용: 코드·슬라이드·공유 노트북에 키를 직접 쓰지 않는다. Colab 보안 비밀(Secrets)에 저장하고, 유출되면 즉시 재발급한다.
호출 예의	호출 제한을 지키고, 짧은 간격으로 반복 호출하지 않는다. (코드에 time.sleep(1) 이상 넣기)
약관·robots.txt	허용된 범위에서만 수집한다. 기사 본문 등은 앞의 3 원칙(저작권)을 함께 확인한다.
차단되면 멈춤	접속 거부·403·429 가 나오면 우회하지 않고 멈춘다. Colab 은 구글 클라우드 주소로 접속하므로 내 PC 에서는 되는데 Colab 에서만 막히는 사이트도 있다. 그럴 땐 다른 출처로 바꾼다.
기록	출처 URL · 수집일시 · 검색 조건을 열로 남겨 팩트체크 때 다시 확인한다.

B-3. 노트북의 표준 3 셀 구조

셀	역할	막혔을 때
셀 1 준비	라이브러리 설치·불러오기, 한글 폰트 등 환경 설정 (필요 없으면 생략)	글자가 네모(□)로 깨짐 → 런타임 다시 시작 후 전체 재실행
셀 2 설정 + 수집	주소·검색어(API 라면 인증키·날짜) 설정 → 내려받기·호출 → 표로 정리	응답 코드(200 이 정상)부터 확인. API 라면 키 값 → 승인 상태 → 호출 한도 순으로 점검
셀 3 결과 확인	CSV 저장 · 시각화 · 핵심 수치 요약	저장 경로(/content/)와 파일 목록 확인

> **에러가 나면**
> 셀을 하나씩 다시 실행하지 말고 메뉴에서 **런타임 → 모두 실행(Run all)** 으로 처음부터 다시 실행합니다. 중간 셀만 다시 실행하면 이전 결과가 남아 원인을 찾기 어렵습니다. "몇 번 셀에서 멈췄나?"를 먼저 확인하세요.

B-4. 준비 — 셀 1 (공통)

아래 셀 1 을 새 Colab 노트북 맨 위에 붙여넣고 실행합니다. 실습 1~3 은 이 셀을 공유하므로 한 번만 실행하면 됩니다.

```
# 셀 1 준비 (노트북을 열 때 한 번만 실행)
!pip -q install feedparser
import time
import requests
import pandas as pd
import feedparser
from io import StringIO
from datetime import datetime
from urllib.parse import quote, urljoin
from bs4 import BeautifulSoup

HEADERS = {"User-Agent": "Mozilla/5.0 (class-practice)"}   # 누가 접속하는지 밝히는 이름표
NOW = datetime.now().strftime("%Y-%m-%d %H:%M")       # 수집일시: 모든 결과에 함께 저장
```

- **!pip -q install feedparser** — RSS 를 읽는 도구를 설치합니다. (Colab 에 없을 수 있어 설치 한 줄을 넣습니다)
- **HEADERS** — 사이트에 "누가 접속하는지" 알리는 이름표입니다. 이름표 없이 접속하면 막는 사이트가 있습니다. 이름표에는 한글을 쓸 수 없습니다(영문·숫자만).
- **NOW** — 수집일시를 한 번 만들어 모든 행에 같은 값으로 저장합니다. 규칙의 '기록' 을 지키는 방법입니다.

B-5. 실습 1 — RSS 로 뉴스 헤드라인 모으기

구글 뉴스 RSS 는 검색어를 넣으면 최신 기사의 제목·링크·언론사·발행일을 목록으로 줍니다. 구글시트의 IMPORTFEED 와 같은 일을 파이썬으로 하고, 20 건을 표로 정리해 CSV 로 저장합니다.

```
# 셀 2 설정 + 수집: 구글 뉴스 RSS로 헤드라인 모으기
KEYWORD = "미세먼지"             # 내 도메인에 맞는 검색어로 바꿔 보세요
rss_url = ("https://news.google.com/rss/search?q=" + quote(KEYWORD)
           + "&hl=ko&gl=KR&ceid=KR:ko")

feed = feedparser.parse(rss_url)
print("가져온 기사 수:", len(feed.entries))

rows = []
for e in feed.entries[:20]:
    rows.append({
        "검색어": KEYWORD,
        "헤드라인": e.title,
        "기사링크": e.link,
        "발행일": e.get("published", ""),
        "언론사": e.get("source", {}).get("title", ""),
        "수집일시": NOW,
    })
df = pd.DataFrame(rows)
display(df.head())

# 셀 3 결과 확인: CSV로 저장
df.to_csv("news_rss.csv", index=False, encoding="utf-8-sig")
print("저장 완료: news_rss.csv")
```

몇 건을 저장할까? — 셀 2의 for 줄이 저장 건수를 정합니다. 구글 뉴스 RSS는 한 번에 최대 100건 정도를 주며, 건수는 검색어와 시점에 따라 달라질 수 있습니다. 가져온 기사 수는 실행하면 "가져온 기사 수"로 확인합니다.

목적	for 줄	저장 건수
앞의 20건만 (교재 기본)	for e in feed.entries[:20]:	최대 20건
가져온 전부	for e in feed.entries:	가져온 기사 수 전부 (보통 100건)
내가 정한 건수	for e in feed.entries[:50]:	숫자만 바꾸면 됨. 가져온 수보다 크면 전부 저장

- **전부 저장하는 법** — 셀 2에서 [:20]만 지우고 for e in feed.entries: 로 만든다. 줄을 새로 붙여 넣을 때 for e in feed.entries가 두 번 겹치지 않게 주의한다. (겹치면 SyntaxError)
- **100건보다 많이 필요할 때** — RSS는 한도가 있으므로 검색어를 바꿔 여러 번 수집해 합치거나, 페이지를 넘기며 받을 수 있는 공식 API(Part C)를 쓴다.

실행 결과 화면 — 셀을 실행하면 아래처럼 "가져온 기사 수", 표(5행 미리보기), "저장 완료" 메시지가 나옵니다. 가져온 기사는 100건이지만 코드가 앞의 20건만 저장합니다. 기사 내용은 검색할 때마다 달라집니다.

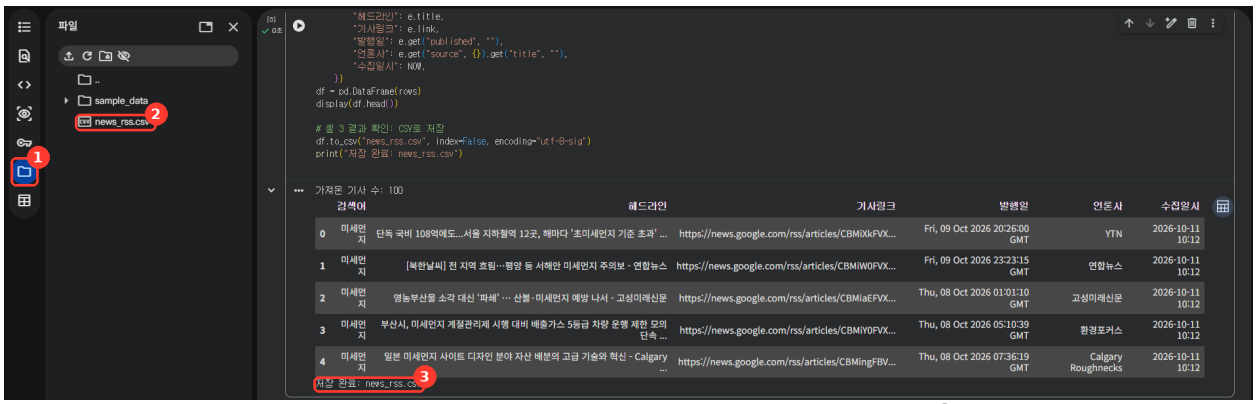


그림 B-1. Colab 실행 결과 — ① 왼쪽 파일(📁) 아이콘 ② 저장된 news_rss.csv ③ 저장 완료 메시지

Colab 에서 CSV 확인하기 — 저장한 파일이 정말 만들어졌는지, 내용이 맞는지 Colab 안에서 바로 볼 수 있습니다.

1. **파일 목록으로 보기** — 왼쪽 📁 아이콘 → /content 폴더에서 news_rss.csv 가 보이는지 확인한다. 안 보이면 새로고침(↻) 아이콘을 누른다.
2. **파일을 열어 보기** — news_rss.csv 를 더블클릭하면 오른쪽에 내용이 열린다. (글자가 쉼표로 이어진 원본 모양으로 보인다)
3. **표로 보기** — 새 셀에 아래 두 줄을 넣고 실행하면 저장된 CSV 를 표로 다시 읽어 보여 준다.

```
df2 = pd.read_csv("news_rss.csv")
display(df2)
```

4. **한 줄 확인** — 셀에 !head -3 news_rss.csv 를 실행하면 첫 3 줄만 글자로 보여 준다.

내 컴퓨터로 내려받기 — Colab 의 파일은 클라우드에 있으므로, 엑셀로 열려면 내려받아야 합니다.

1. 왼쪽 📁 아이콘을 눌러 파일 창을 연다.
2. news_rss.csv 오른쪽의 ⋮(점 3 개)를 누르고 **다운로드**를 선택한다. 파일이 브라우저의 다운로드 폴더에 저장된다.
3. 엑셀로 열면 한글이 정상으로 보인다. (저장할 때 encoding="utf-8-sig"를 썼기 때문이다) 구글시트는 파일 → 가져오기 → 업로드로 불러온다.

코드로 내려받으려면 새 셀에 아래 두 줄을 실행합니다. 브라우저가 파일 저장을 시작합니다.

```
from google.colab import files
files.download("news_rss.csv")
```

> **파일은 임시로만 보관됩니다**
> /content 폴더는 임시 저장 공간입니다. 노트북 연결이 끊기거나 세션이 끝나면 CSV 가 사라집니다. 실습이 끝나기 전에 꼭 내려받으세요. 계속 보관하려면 왼쪽 파일 창의 드라이브 마운트(폴더 모양 아이콘)로 내 구글 드라이브를 연결하고, 저장 경로를 /content/drive/MyDrive/news_rss.csv 처럼 바꿉니다. 실습 2·3 의 CSV(wiki_table.csv, list_scrape.csv)도 같은 방법으로 확인하고 내려받습니다.

본문은 가져오지 않습니다

RSS 는 제목·링크·출처까지만 줍니다. 앞의 3 원칙에서 언론사 기사는 “헤드라인 1 개 + 출처 표기” 까지만 허용하는 것과 같은 범위입니다. 기사 본문이 필요하면 공공기관 사이트(공공누리 1·2 유형)에서 수집합니다. 구글 뉴스 주소가 막히면 정책브리핑이나 다른 언론사의 RSS 주소로 바꿔 실습해도 됩니다.

코드 읽기 연습 — 아래 질문에 답해 보세요.

- KEYWORD 를 바꾸면 URL 의 어느 부분이 바뀌나요? quote()는 왜 필요할까요?
- feed.entries[:20]에서 [:20]은 무슨 뜻인가요?
- Part A 의 IMPORTFEED 와 비교해, 결과를 CSV 로 저장하는 일은 어느 쪽이 더 쉬웠나요?

내 답과 실행 결과 메모 (가져온 기사 수 · 내가 쓴 검색어):

B-6. 실습 2 — 웹페이지의 표 가져오기

페이지 안의 표는 pandas.read_html 한 줄로 전부 가져옵니다. 구글시트의 IMPORTHTML 과 같은 일입니다. 그런데 한 페이지에 표가 열 개씩 잡히기도 해서, 어떤 표가 필요한 표인지 가려내는 일이 더 중요합니다. 이 실습은 아래 3 단계로 진행합니다.

단계	할 일	누가
1 단계	셀 2 를 실행해 페이지 안의 모든 표를 번호·크기·앞 3 행으로 출력한다	Colab
2 단계	출력 결과를 복사해 AI 에게 붙여 넣고, 내 목적에 맞는 표 번호를 확인받는다	AI + 나
3 단계	AI 가 알려 준 번호의 표만 셀 3 에서 가져와 저장한다. 나머지 표는 쓰지 않는다	Colab

1 단계. 모든 표 출력하기 — 셀 2

```
# 셀 2 설정 + 수집: 웹페이지의 표 가져오기
URL = "https://ko.wikipedia.org/wiki/대한민국의_행정_구역"   # 표가 있는 다른 문서로 바꿔도 됩니다

res = requests.get(URL, headers=HEADERS, timeout=10)
print("응답 코드:", res.status_code)              # 200 이면 정상

tables = pd.read_html(StringIO(res.text))         # 페이지 안의 표를 전부 리스트로 가져온다
print("표 개수:", len(tables))
for i, t in enumerate(tables):                    # 모든 표를 번호·크기·앞 3 행으로 훑어보기
    print("표 번호", i, "크기(행, 열):", t.shape)
    display(t.head(3))
```

실행 결과 읽는 법 — 셀을 실행하면 아래와 같은 글자가 나옵니다. 번호 순서대로 읽어 보세요.

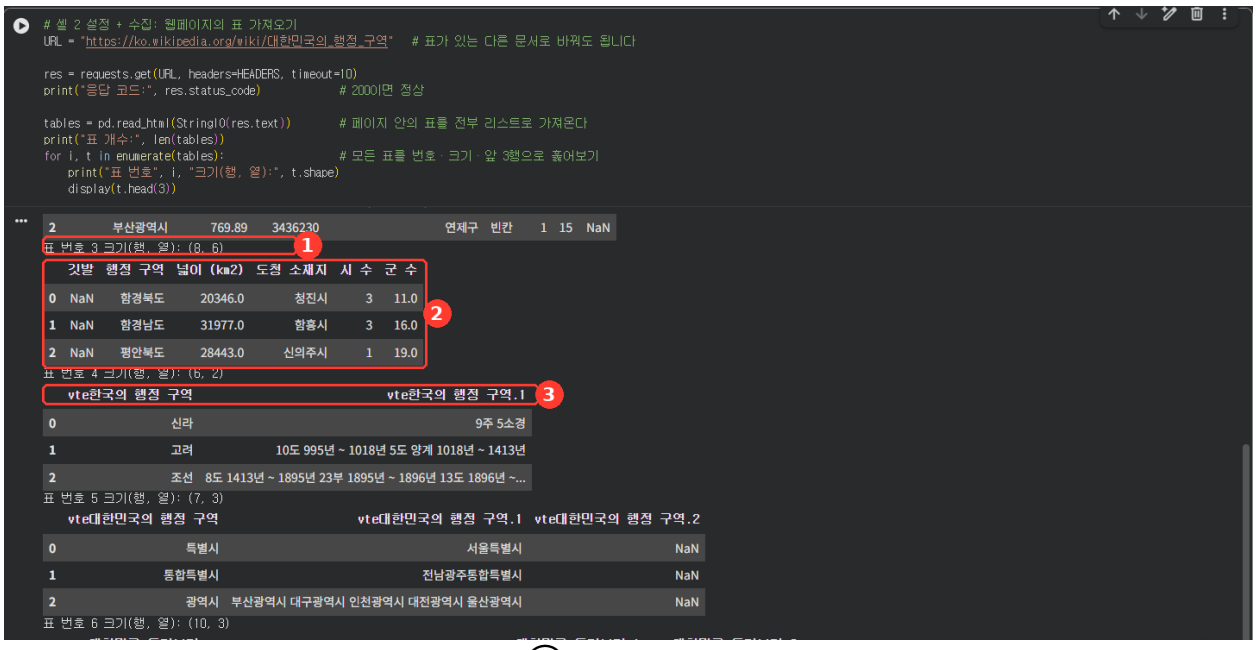


그림 B-2. 셀 2 실행 화면(출력 가운데 부분) — ① 표 번호와 크기 ② 표의 앞 3 행 ③ vte 로 시작하는 분류 틀

번호	화면에 나온 글	읽는 법
-	응답 코드: 200 · 표 개수: 10	출력의 맨 위(화면을 위로 올리면 보입니다)에 나옵니다. 응답 코드 200 은 사이트가 페이지를 정상으로 줬다는 뜻이고, 403·429 는 접속이 막혔다는 뜻입니다(Part B-8 표 참고), 404 는 주소가 틀렸다는 뜻입니다. 200 이 아니면 여기서 멈춥니다. 표 개수는 pandas 가 찾은 표의 전체 수이며, 안내 상자나 분류 틀도 표 모양이라 눈으로 센 것보다 많을 수 있습니다.
①	표 번호 3 크기(행, 열): (8, 6)	표마다 한 줄씩 나옵니다. 3 은 표의 번호이고 0 부터 시작합니다. (8, 6)은 8 행 6 열이라는 뜻이며, 제목 줄은 행 수에 포함되지 않습니다.
②	깃발 · 행정 구역 · 넓이 · 도청 소재지 … / 함경북도 20346.0 …	그 표의 앞 3 행입니다. 열 이름과 첫 내용을 보면 어떤 표인지 알 수 있습니다. 이 표는 북한의 도 표입니다. NaN 은 값이 비어 있다는 뜻입니다.
③	vte 한국의 행정 구역	열 이름이 vte 로 시작하는 표는 위키백과 문서 아래쪽의 분류 틀(보기·편집용 상자)입니다. 데이터 표가 아니라는 신호입니다.

2 단계. 결과를 AI 에게 확인시키기

1 단계의 출력 글자 전체를 복사해 AI(Claude 등)에게 붙여 넣고 아래처럼 묻습니다. 표 개수가 많거나 크기가 큰 페이지는 눈으로 고르기 어려워서 AI 에게 먼저 확인시키면 빠릅니다.

```
아래는 파이썬 pd.read_html 로 위키백과 "대한민국의 행정 구역" 문서에서 가져온 모든 표의 번호, 크기(행, 열), 앞 3 행입니다.
내 목적은 17 개 시·도의 넓이와 인구를 정리하는 것입니다.
1) 쓸 만한 표의 번호를 알려 주세요.
2) 나머지 표를 쓰지 않는 이유를 한 줄씩 알려 주세요.
3) 가져온 뒤에 정제가 필요해 보이는 곳이 있으면 알려 주세요.
```

```
(여기에 출력 결과를 붙여 넣기)
```

AI 는 표 번호와 이유를 알려 줍니다. 이번 위키 문서에서는 대략 아래와 같은 답이 나옵니다. (예시이며, AI 의 답은 매번 조금씩 다릅니다)

표 번호	크기	앞부분 내용	판단
0	(1, 2)	“이 문서의 일부는 오래된 정보를 가지고 있어 …”	안내 상자. 쓰지 않음
1	(10, 1)	대한민국의 행정 구역 / 광역자치단체 …	제목·분류 상자. 쓰지 않음
2	(17, 8)	행정 구역·넓이·인구·시·도청 소재지·시·군·구 (서울특별시, 부산광역시 …)	쓸 표 (17 개 시·도 데이터)
3	(8, 6)	함경북도·함경남도·평안북도 … (도청 소재지, 시 수, 군 수)	북한 행정 구역 표. 목적이 다르면 쓰지 않음
4~8	5~10 행, 열 2~3 개	열 이름이 vte 로 시작, 신라·고려·조선 / 특별시·광역시 / 독립국·속령	문서 아래쪽 분류 틀(내비게이션 상자). 쓰지 않음

AI 의 답도 사람이 확인합니다

AI 는 출력된 글자만 보고 판단하므로 틀릴 수 있습니다. 3 원칙의 “사람의 검수” 에 따라, 고른 번호의 표를 3 단계에서 직접 열어 내용이 맞는지 눈으로 확인합니다. 출력 결과에는 공개된 페이지의 표만 있으므로 붙여 넣어도 되지만, 개인정보가 들어 있는 표는 AI 에 붙여 넣지 않습니다.

3 단계. 필요한 표만 가져와 저장하기 — 셀 3

```
# 셀 3 결과 확인: AI 가 알려 준 표만 CSV 로 저장
picks = {2: "wiki_sido.csv", 3: "wiki_north.csv"}   # {표 번호: 저장할 파일명}

for no, filename in picks.items():
    df = tables[no].copy()
    df["출처 URL"] = URL
    df["수집일시"] = NOW
    df.to_csv(filename, index=False, encoding="utf-8-sig")
    print("저장 완료:", filename, df.shape)
    display(df.head())
```

- **picks** 에 AI 가 알려 준 표 번호와 저장할 파일명을 적습니다. 한 개만 저장하려면 {2: "wiki_sido.csv"} 처럼 한 줄만 두고, 여러 개이면 쉼표로 이어 적습니다. 나머지 표는 따로 지울 필요 없이 쓰지 않으면 되고, CSV 에는 고른 표만 저장됩니다.

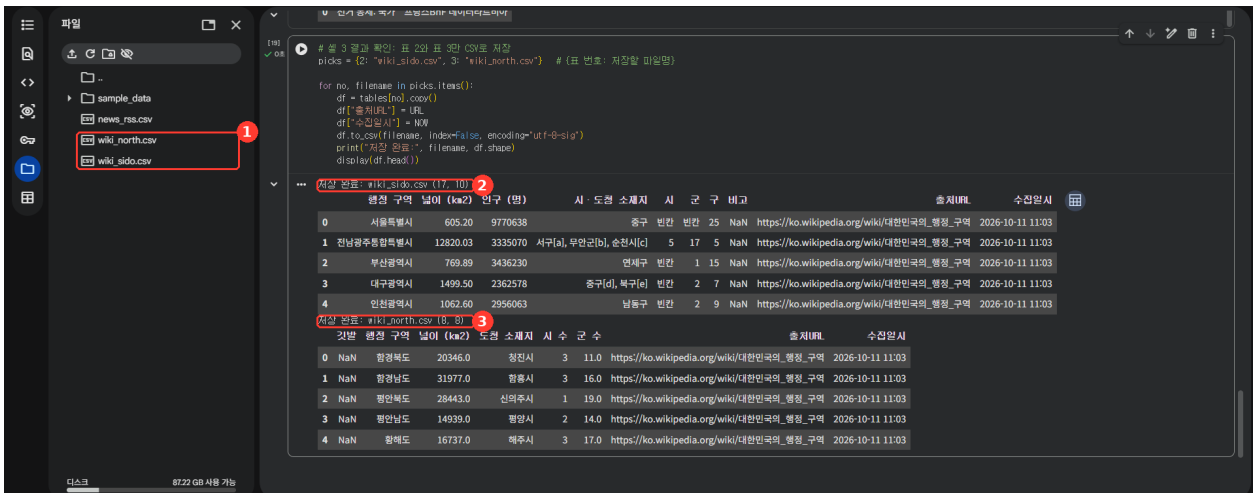


그림 B-3. 셀 3 실행 결과 — ① 왼쪽 파일 창에 생긴 wiki_sido.csv, wiki_north.csv ② 시·도 표 저장 완료 (17, 10) ③ 북한 도 표 저장 완료 (8, 8)

- **(17, 10)과 (8, 8)** — 저장 완료 줄의 괄호는 (행, 열)입니다. 원래 표가 8 열과 6 열이었는데 출처 URL 과 수집일시 열이 더해져 10 열과 8 열이 되었습니다. 열이 2 개 늘었는지 확인하면 저장이 제대로 됐다는 것을 알 수 있습니다.
- 저장된 CSV 를 컴퓨터로 받는 방법은 B-5 의 “내 컴퓨터로 내려받기”와 같습니다. 파일 창에서 파일 오른쪽 ⋮ → 다운로드를 누릅니다. 세션이 끝나면 파일이 사라지므로 실습이 끝나기 전에 내려받습니다.
- 출력된 첫 5 행이 AI 의 설명과 맞는지 확인합니다. 아니면 picks 의 번호를 바꿔 다시 실행합니다.
- 2 번 표를 열어 보면 데이터를 쓰기 전에 정제할 곳이 보입니다. 시·군 열의 “빈칸” 글자, “서구[a]”처럼 붙은 각주 표시, 비고 열의 NaN(빈 값)입니다. 이 정리는 Part E(정제)에서 다룹니다.
- 위키백과 문서는 사람들이 고치므로 표 순서가 바뀔 수 있습니다. 번호를 외우지 말고 실행할 때마다 1~2 단계로 확인합니다.
- “No tables found” 오류는 그 페이지에 표가 없거나, 자바스크립트로 나중에 그려지는 표라는 뜻입니다.
- 저장할 때 출처 URL 과 수집일시 열을 함께 남깁니다.

내가 고른 표 번호 · 표 크기 · 어떤 내용의 표였나:

B-7. 실습 3 — 목록 페이지에서 헤드라인·링크 뽑기

Part A 에서 IMPORTXML 로 수집한 목록 페이지를 파이썬으로 다시 수집해 봅니다. 같은 사이트를 쓰면 시트와 파이썬의 결과를 비교할 수 있습니다. 먼저 robots.txt 로 수집이 허용되는지 확인하고, 허용될 때만 진행합니다.

선택자(CSS) 얻기 — 구글시트의 XPath 와 같은 방법입니다.

1. 수집할 목록 페이지를 크롬으로 열고, 기사 제목 하나를 우클릭 → **검사**를 누른다.
2. 개발자도구에서 강조된 줄이 링크(a 태그)인지 확인한다. 제목 글자만 감싼 안쪽 태그라면 한 칸 위의 a 태그를 고른다.
3. 그 줄을 우클릭 → **Copy → Copy selector**.
4. 복사한 선택자에서 :nth-child(1)처럼 순번이 붙은 부분을 지우면 목록 전체를 가져온다.

```
# 셀 2 설정 + 수집: 목록 페이지에서 헤드라인·링크 뽑기
```

```
LIST_URL = "목록 페이지 주소를 붙여 넣기"
CSS = "Copy selector로 복사한 선택자를 붙여 넣기"
FILENAME = "list_scrape.csv"                  # 저장할 파일 이름 (사이트마다 바꾸기)

# 1) 수집이 허용되는지 먼저 확인 (robots.txt)
import urllib.robotparser as rp
base = "/".join(LIST_URL.split("/")[:3])
robot = rp.RobotFileParser(base + "/robots.txt")
robot.read()
allowed = robot.can_fetch("*", LIST_URL)
print("수집 허용?", allowed)
assert allowed, "robots.txt가 허용하지 않아 여기서 멈춥니다"

# 2) 목록 페이지 내려받기
res = requests.get(LIST_URL, headers=HEADERS, timeout=10)
res.encoding = res.apparent_encoding            # 한글이 깨지지 않게
print("응답 코드:", res.status_code)
soup = BeautifulSoup(res.text, "html.parser")

# 3) 선택자로 헤드라인과 링크 뽑기
rows = []
for a in soup.select(CSS):
    href = a.get("href")
    if not href:
        continue
    rows.append({
        "헤드라인": a.get_text(strip=True),
        "기사링크": urljoin(LIST_URL, href),       # 일부 주소를 완전한 주소로
        "목록URL": LIST_URL,
        "수집일시": NOW,
    })
df = pd.DataFrame(rows)
print(len(df), "건")
display(df.head())

# 셀 3 결과 확인: CSV로 저장
df.to_csv(FILENAME, index=False, encoding="utf-8-sig")
print("저장 완료:", FILENAME)
```

셀 2 맨 위의 세 줄만 내 것으로 바꿉니다. **LIST_URL** 의 따옴표 안에 목록 페이지 주소를, **CSS** 의 따옴표 안에 복사한 선택자를, **FILENAME** 에 저장할 파일 이름을 적습니다. 따옴표는 지우지 않고, 따옴표 안의 안내 글만 바꿉니다. 아래는 연습용 예시 값이고, 두 예시 모두 Colab 에서 수집이 확인되었습니다. 이 값으로 먼저 해 보고, 다음에는 내가 고른 사이트의 주소와 선택자를 넣어 봅니다.

바꿀 줄	예시 1 — 연합뉴스 정치	예시 2 — 연합뉴스 문화 (그림 B-4)
LIST_URL (주소)	https://www.yna.co.kr/politics/all	https://www.yna.co.kr/culture/all
CSS (선택자)	#container > div:nth-of-type(2) > div:nth-of-type(2) > div:nth-of-type(1) > section > div > ul > li > div > div > strong > a	#container > div.container521 > div.content03 > div.section01 > section > div > ul > li > div > div > strong > a

바꿀 줄	예시 1 — 연합뉴스 정치	예시 2 — 연합뉴스 문화 (그림 B-4)
FILENAME (파일 이름)	yna_politics.csv	yna_culture.csv

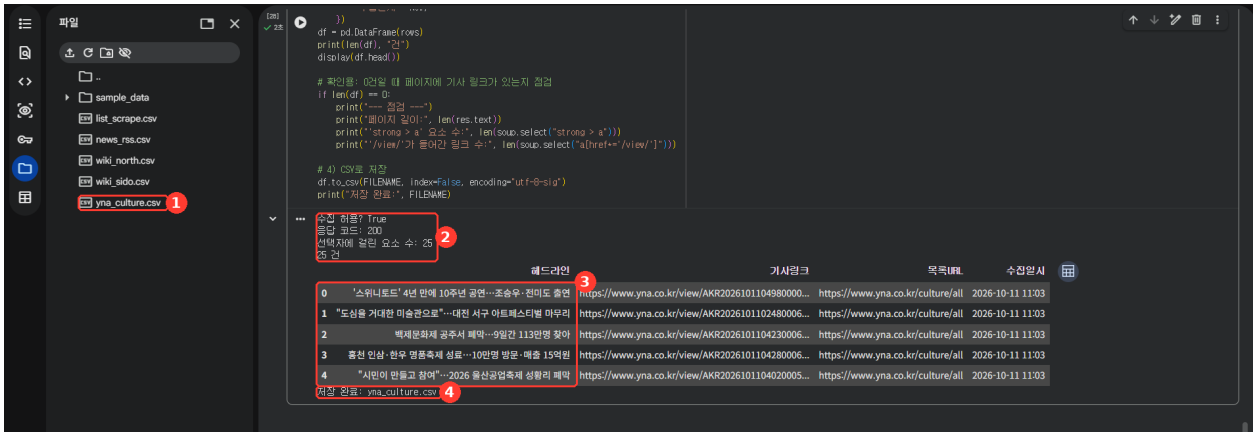


그림 B-4. 연합뉴스 문화 목록 수집 결과 — ① 저장된 yna_culture.csv ② 수집 허용 True · 응답 200 · 선택자에 걸린 요소 25 · 25 건 ③ 헤드라인·기사링크·목록 URL·수집일시 ④ 저장 완료

- **수집 허용? True** 는 robots.txt 가 이 주소의 자동 수집을 막지 않았다는 뜻입니다. False 이면 코드가 AssertionError 로 멈추고, 그 사이트는 코드로 수집하지 않습니다.
- **25 건**은 선택자에 걸린 링크의 수입니다. 목록 한 페이지의 기사 수와 같아야 정상입니다. (정치·문화 모두 25 건) 0 건이면 선택자가 틀렸거나 자바스크립트로 나중에 그려지는 페이지입니다.
- 표의 **기사링크**는 urljoin 으로 완전한 주소가 된 것입니다. 목록 URL 과 수집일시를 함께 저장해 출처를 남깁니다.

> **연합뉴스를 쓸 때의 범위**
> 연합뉴스는 3 원칙에서 AI 활용이 제한된 출처입니다. 이 실습은 헤드라인·기사링크·출처·수집일시까지만 모으고, 기사 본문은 가져오지 않습니다. 기사 끝의 AI 학습·활용 금지 문구도 확인합니다.

> **경고가 나오거나 0 건이 나오면**
> 허용 여부가 False 이면 거기서 멈추고 다른 사이트를 고릅니다. 응답 코드가 200 인데 0 건이면 선택자가 틀렸거나 자바스크립트로 나중에 그려지는 페이지입니다. 억지로 우회하지 않습니다.

선택자가 안 맞을 때 (0 건) — 실제 사례

- 같은 연합뉴스라도 분야마다 페이지 구조가 다릅니다. 정치 목록에서 된 선택자가 문화 목록에서는 0 건이 나왔습니다.
- 크롬에서 복사한 선택자의 **li:nth-child(1)** 은 첫 기사 하나만 가리킵니다. 목록 전체를 가져오려면 **li** 로 고칩니다.
- 복사한 선택자가 **span** 처럼 글자만 감싼 안쪽 태그에서 끝나면, 링크(href)가 없어 모두 건너뛰어 0 건이 됩니다. 링크가 있는 **a** 에서 끝내도록 고칩니다.
- 그래도 안 되면 **strong > a** 처럼 짧은 선택자로 줄여 보고, 아래 점검 코드로 어느 구역에서 잡히는지 확인합니다. 메뉴 글자나 다른 목록이 섞여 있으면 구역 이름(class)을 넣어 좁힙니다.

```
# 점검: strong > a 로 잡힌 링크가 어느 구역에 있는지 확인
for i, a in enumerate(soup.select("strong > a")):
    box = a.find_parent(lambda t: t.name in ("section", "div", "ul") and (t.get("id")
```

```
or t.get("class")))
    where = (box.name + "#" + str(box.get("id")) + "." + ".".join(box.get("class",
[]))) if box else "?"
    print(i, a.get_text(strip=True)[:40], "|", where)
```

코드 읽기 연습 — Part A 와 비교해 보세요.

- Part A 의 XPath 는 코드의 어느 부분으로 바뀌었나요?
- urljoin 은 Part A 에서 "기본 주소를 앞에 붙인다" 고 했던 어떤 작업을 대신하나요?
- 시트와 파이썬이 가져온 건수가 다르다면 어디부터 확인하겠습니까?

내가 시험한 사이트 · robots.txt 허용 여부 · 가져온 건수 · 시트 결과와 차이:

B-8. 막혔을 때

증상	원인	해결
응답 코드가 403 · 429, 또는 접속 거부	사이트가 자동 접속을 막거나 호출이 너무 잦음	우회하지 않고 멈춘다. HEADERS 와 time.sleep 을 확인하고, 그래도 막히면 다른 출처로 바꾼다
응답은 200 인데 결과가 0 건	선택자가 틀렸거나, 자바스크립트로 나중에 그려지는 페이지	크롬에서 Copy selector 를 다시 복사한다. 그래도 0 건이면 이 사이트는 공식 API 나 다른 출처를 쓴다
한글이 깨져 보임	페이지의 글자 인코딩을 잘못 읽음	res.encoding = res.apparent_encoding 줄을 넣는다
링크가 ?menukey=...처럼 일부만 나옴	상대 주소	urljoin(목록주소, href)으로 완전한 주소로 만든다
No tables found	그 페이지에 표가 없음, 또는 자바스크립트로 그려지는 표	다른 페이지로 바꾸거나 HTML 에 table 태그가 있는지 확인
UnicodeEncodeError: latin-1 codec can't encode	HEADERS 의 User-Agent 에 한글을 넣음 (헤더는 영문·숫자만 가능)	User-Agent 값을 "Mozilla/5.0 (class-practice)"처럼 영문으로 바꾸고 셀 1 을 다시 실행
ModuleNotFoundError	셀 1 을 실행하지 않았거나 설치가 안 됨	런타임 → 모두 실행으로 셀 1 부터 다시 실행

내가 만난 오류와 해결 방법:

시트와 파이썬, 언제 무엇을 쓰나

한두 페이지를 빨리 확인할 때는 구글시트(IMPORTXML). 여러 페이지를 반복하거나 정리·저장까지 한 번에 할 때는 Colab 파이썬. 어느 쪽이든 먼저 수집이 허용되는 출처인지, AI 로 정제해도 되는 출처인지(3 원칙)를 확인합니다. 파이썬으로 모은 CSV 를 Part E 의 정제에 쓰려면, CSV 를 구글시트로 열어 6 열 양식(구분·분야·URL·헤드라인·기사링크·기사원문)에 맞춥니다. 기사원문은 공공기관 사이트에서 Part A 의 방법으로 채웁니다.

Part C. Colab 으로 API 호출해 수집하기 (인증키 사용)

Part B 의 방법으로는 가져오기 어려운 데이터, 구조가 정해진 데이터, 대량·반복 수집은 공식 API 로 가져옵니다. API 는 인증키가 필요합니다. 코드는 AI 와 함께 작성하더라도, 각 줄이 무슨 일을 하는지 직접 읽고 실행해 결과를 확인합니다. 코딩 수집의 규칙(B-2)과 노트북 3 셀 구조(B-3)는 그대로 적용합니다.

C-0. API 와 인증키 — 무엇이고, 왜 필요하고, 어떻게 받나

API 란? 기관이 "이 주소로 이런 조건을 담아 요청하면, 이 형식으로 데이터를 보내 드립니다"라고 공식으로 열어 둔 데이터 창구입니다. 웹페이지가 사람이 눈으로 보라고 만든 화면이라면, API 는 프로그램이 바로 읽을 수 있게 정리된 데이터(주로 JSON)를 줍니다. Part B 에서 화면에서 필요한 부분을 골라냈다면, 여기서는 처음부터 정리된 데이터를 받습니다.

구분	웹페이지에서 가져오기 (Part B)	공식 API (Part C)
받는 것	사람이 보라고 만든 화면(HTML)에서 필요한 부분을 골라냄	프로그램이 읽기 좋게 정리된 데이터 (JSON 등)를 바로 받음
안정성	사이트 디자인이 바뀌면 선택자가 맞지 않게 될 수 있음	주소와 항목 이름이 문서로 정해져 있어 상대적으로 안정적
허용 범위	robots.txt·약관을 내가 직접 확인해야 함	기관이 공식으로 열어 둔 통로이고, 이용 조건이 발급 때 안내됨
조건 지정	화면에 보이는 만큼만 가져옴	날짜·지역·검색어 등 조건을 정해 정확히 요청
인증키	필요 없음	필요함

인증키란? 요청을 보낸 사람이 누구인지 알려 주는 문자열입니다. 기관은 키가 있는 요청에만 데이터를 보냅니다. 키 없이 요청하면 데이터 대신 오류 메시지가 돌아옵니다.

왜 키가 필요한가?

- **누가 얼마나 쓰는지 관리하기 위해서입니다.** 키별로 호출 횟수를 세어 일일 한도를 정하고, 한 사람의 과도한 호출로 서버가 느려지는 것을 막습니다.
- **이용 조건에 동의한 사용자에게만 제공하기 위해서입니다.** 출처 표시, 상업적 이용 여부 같은 조건을 발급 때 확인하고 동의합니다.
- **문제가 생기면 그 키만 멈추고 확인할 수 있기 때문입니다.** 비정상적인 호출이 있으면 해당 키만 제한할 수 있습니다.

왜 우리가 키를 받아 쓰는가? 키를 받아 쓰는 것은 기관이 공식으로 허락한 통로로 데이터를 받는다는 뜻입니다. 3 원칙의 ① 합법적 수집 구분을 가장 확실하게 지키는 방법이고, 출처와 기준 시점이 분명해서 팩트체크 때 다시 확인하기도 쉽습니다.

서비스마다 키 이름과 넣는 자리가 다릅니다. 같은 "인증키"여도 코드에서 부르는 이름이 다르므로, 새 API 를 쓸 때는 해당 기관의 안내 문서를 먼저 확인합니다.

서비스	발급받는 곳	키 이름	코드에서 넣는 자리
영화진흥위원회 KOBIS	kobis.or.kr/kobisopenapi	인증키	params 의 key

서비스	발급받는 곳	키 이름	코드에서 넣는 자리
공공데이터포털 (에어코리아·기상청 ASOS 등)	data.go.kr	일반 인증키 (Encoding·Decoding 두 종류)	params 의 serviceKey (requests 에는 Decoding 키)
국가통계포털 KOSIS	kosis.kr/openapi	인증키	params 의 apiKey
기상청 API 허브	apihub.kma.go.kr	인증키	params 의 authKey

발급 방법 (KOBIS 기준)

1. kobis.or.kr/kobisopenapi 에 접속해 로그인한다. 처음이면 회원가입을 먼저 한다.
2. 상단의 **키 발급/관리** 메뉴에서 **오픈 API 키 발급받기**를 누른다.
3. 신청 화면의 안내에 따라 사용 목적 등을 입력하고, 이용 약관을 읽은 뒤 동의하고 신청한다.
4. 발급된 키를 확인하고 복사한다.
5. 복사한 키를 아래 “키 보관하기” 방법으로 Colab 에 저장한다.

다른 서비스도 흐름은 같습니다. **가입 → 사용할 API 선택·활용 신청 → 키 확인 → 한 건만 호출해 보기**입니다. 서비스마다 화면과 승인 방식(즉시 또는 심사 후)이 다릅니다. 메뉴 이름이 다르면 그 사이트의 이용안내나 FAQ 를 확인합니다.

> **키를 복사할 때 자주 생기는 문제**
> 키 앞뒤에 공백이 따라 들어가거나, 끝의 “=” 같은 문자가 잘리면 인증 오류가 납니다. 복사한 뒤 키 전체가 그대로 들어갔는지 확인하고, 안 되면 다시 복사합니다.

키 보관하기 (Colab 보안 비밀) — 키를 코드에 직접 적으면 노트북을 공유할 때 함께 노출됩니다. Colab 의 보안 비밀에 저장하고 코드에서는 이름으로 불러옵니다.

1. Colab 왼쪽의 🔑**(보안 비밀)** 아이콘을 누른다.
2. **새 보안 비밀 추가**를 눌러 이름에 KOBIS_KEY, 값에 발급받은 키를 붙여 넣는다.
3. **노트북 액세스** 스위치를 켠다.
4. 코드에서는 아래 두 줄로 불러온다.

```
from google.colab import userdata
KOBIS_KEY = userdata.get("KOBIS_KEY")   # 키를 코드에 직접 쓰지 않는다
```

C-1. API 키 발급과 주의점

1. 서비스(영화진흥위원회 KOBIS, 네이버 개발자센터, 공공데이터포털 등)에 **가입·본인 인증**한다.
2. 사용할 API 를 골라 **활용 신청**한다. 승인이 필요한 서비스는 미리 신청한다.
3. 발급된 **인증키**를 확인한다. (서비스마다 이름이 다름: Client ID·Secret, 인증키, OC 값)
4. Colab 에서 **한 건만 호출**해 본다. 안 되면 승인 상태 → 키 값 → 한도 순으로 점검한다.

> **키는 내 비밀번호처럼**
> 키는 개인 계정에 묶여 있으므로 공용 사용·전달 금지, 서비스마다 일일 한도가 있으니 한 건 테스트 후 간격을 두고 반복 호출, 수집 결과의 실명 등 개인정보는 필요한 항목만 저장합니다.

C-2. 실습 1 — 일일 박스오피스 TOP10 가져오기

KOBIS API 로 어제 날짜의 일일 박스오피스를 받아 CSV 로 저장합니다. 셀 하나에 설정 → 호출 → 출력 → 저장이 모두 들어 있습니다.

```
import requests
import csv
from datetime import datetime, timedelta
from google.colab import userdata

KOBIS_KEY = userdata.get("KOBIS_KEY")      # 키를 코드에 직접 쓰지 않는다
TARGET_DATE = (datetime.now() - timedelta(days=1)).strftime("%Y%m%d")   # 어제

url =
"http://kobis.or.kr/kobisopenapi/webservice/rest/boxoffice/searchDailyBoxOfficeList.js
on"
params = {"key": KOBIS_KEY, "targetDt": TARGET_DATE}

response = requests.get(url, params=params, timeout=10)
print("응답 코드:", response.status_code)
data = response.json()

if "boxOfficeResult" not in data:
    print("오류 응답:", data)               # 키 오류·한도 초과 등은 여기에 나옵니다
else:
    box_office_list = data["boxOfficeResult"]["dailyBoxOfficeList"]
    print(f"=== {TARGET_DATE} 일일 박스오피스 TOP 10 ===")
    for movie in box_office_list:
        print(f'{movie["rank"]}위 | {movie["movieNm"]} | 당일 {int(movie["audiCnt"]):,}
명 | 누적 {int(movie["audiAcc"]):,}명')

    csv_filename = f"boxoffice_{TARGET_DATE}.csv"
    with open(csv_filename, "w", newline="", encoding="utf-8-sig") as f:
        writer = csv.writer(f)
        writer.writerow(["순위", "영화제목", "당일관객수", "누적관객수", "당일매출액"])
        for m in box_office_list:
            writer.writerow([m["rank"], m["movieNm"], m["audiCnt"], m["audiAcc"],
m["salesAmt"]])
    print("저장 완료:", csv_filename)
```

실행 결과 — 저장된 boxoffice_20261011.csv 의 내용입니다. 날짜는 실행하는 날에 따라 달라집니다.

순위	영화제목	당일관객수	누적관객수	당일매출액
1	극장판 치이카와: 인어 섬의 비밀	119,961	1,281,485	1,228,387,370
2	암살자(들)	108,047	2,992,544	1,078,446,920
3	타짜: 벨제붑의 노래	53,759	1,744,160	561,682,590
4	오디세이	25,270	12,179,163	351,386,710
5	부활남: 더 레드	15,183	302,355	167,545,100
6	룩백	10,737	62,383	117,600,190

순위	영화제목	당일관객수	누적관객수	당일매출액
7	입에 대한 앙케트	9,349	50,474	96,522,920
8	디거	4,618	70,497	49,193,850
9	옵세션	4,469	955,136	51,585,850
10	극장판 래브라도: 망고축제를 지켜라	4,085	34,805	36,716,600

응답 코드 200 이 나오고 10 행이 출력되면 성공입니다. 키가 잘못되었거나 한도를 넘으면 "오류 응답:" 뒤에 KOBIS 가 보낸 메시지가 나옵니다.

코드 읽기 연습 — 아래 질문에 답해 보세요.

- params 에 들어간 두 값은 각각 무엇을 뜻하나요?
- data["boxOfficeResult"]["dailyBoxOfficeList"]는 JSON 의 어느 부분을 꺼내는 것인가요?
- 파일을 저장할 때 encoding="utf-8-sig"를 쓴 이유는? (힌트: 엑셀에서 열었을 때 한글)

내 답과 실행 결과 메모:

C-3. 실습 2 — 지난 10 일 집계로 확장하기

KOBIS 일일 API 는 날짜를 하나만 받습니다. 그래서 날짜를 어제부터 하루씩 거슬러 올라가며 10 번 호출하고, 받은 TOP10 을 한 표로 합칩니다. 핵심은 **날짜를 바꿔 가며 반복 호출하는 반복문**과 **결과를 리스트에 쌓았다가 표(DataFrame)로 만드는 것**입니다. 기간을 바꾸려면 DAYS 값만 고칩니다.

셀 1 — 10 일치 수집하고 CSV 로 저장

```
import time
import requests
import pandas as pd
from datetime import datetime, timedelta
from google.colab import userdata

KOBIS_KEY = userdata.get("KOBIS_KEY")      # 키를 코드에 직접 쓰지 않는다
DAYS = 10                                   # 어제부터 몇 일치를 받을지
NOW = datetime.now().strftime("%Y-%m-%d %H:%M")

url =
"http://kobis.or.kr/kobisopenapi/webservice/rest/boxoffice/searchDailyBoxOfficeList.json"

rows = []
for i in range(1, DAYS + 1):
    target = (datetime.now() - timedelta(days=i)).strftime("%Y%m%d")
    res = requests.get(url, params={"key": KOBIS_KEY, "targetDt": target}, timeout=10)
    data = res.json()
```

```
    if "boxOfficeResult" not in data:       # 키 오류·한도 초과 등
        print(target, "오류 응답:", data)
        break

    for m in data["boxOfficeResult"]["dailyBoxOfficeList"]:
        rows.append({
            "날짜": target,
            "순위": int(m["rank"]),
            "영화제목": m["movieNm"],
            "당일관객수": int(m["audiCnt"]),
            "누적관객수": int(m["audiAcc"]),
            "당일매출액": int(m["salesAmt"]),
            "수집일시": NOW,
        })
    print(target, "수집 완료:", len(data["boxOfficeResult"]["dailyBoxOfficeList"]),
"건")
    time.sleep(1)                           # 호출 예의: 1초 간격

df = pd.DataFrame(rows)
filename = "boxoffice_last10days.csv"
df.to_csv(filename, index=False, encoding="utf-8-sig")
print("저장 완료:", filename, df.shape)
display(df.head(12))
```

실행 결과 — 맨 아래에 "저장 완료: boxoffice_last10days.csv (100, 7)"이 나옵니다. 10 일 × 10 위 = 100 행, 열은 7 개입니다. 아래는 표의 맨 앞 5 행입니다.

날짜	순위	영화제목	당일관객수	누적관객수	당일매출액
20261011	1	극장판 치이카와: 인어 섬의 비밀	119,961	1,281,485	1,228,387,370
20261011	2	암살자(들)	108,047	2,992,544	1,078,446,920
20261011	3	타짜: 벨제붑의 노래	53,759	1,744,160	561,682,590
20261011	4	오디세이	25,270	12,179,163	351,386,710
20261011	5	부활남: 더 레드	15,183	302,355	167,545,100

- **날짜 열**이 있어야 10 일치를 구분할 수 있습니다. 어제부터 거슬러 올라가므로 가장 최근 날짜가 맨 위에 옵니다.
- 같은 영화가 여러 날짜에 나옵니다. 영화 하나를 골라 보면 날마다 어떻게 변했는지 볼 수 있습니다. 어느 날 TOP10 밖이었던 영화는 그날 행이 없습니다.
- **누적관객수는 개봉일부터의 전체 누적**이라 10 일 합계가 아닙니다. 10 일 동안의 합계를 구하려면 **당일관객수**를 더해야 합니다.
- **수집일시** 열은 코드를 실행한 시각으로, 모든 행에 같은 값이 들어갑니다. 팩트체크 때 언제 가져온 자료인지 확인하는 근거입니다.

셀 2 — 결과를 그래프로 보기 (셀 1 을 실행한 뒤 이어서 실행합니다. df 를 그대로 씁니다.)

```
!apt-get -qq -y install fonts-nanum > /dev/null

import matplotlib.pyplot as plt
import matplotlib.font_manager as fm
from matplotlib.ticker import FuncFormatter
from datetime import datetime

fm.fontManager.addfont("/usr/share/fonts/truetype/nanum/NanumGothic.ttf")
plt.rc("font", family="NanumGothic")
plt.rcParams["axes.unicode_minus"] = False

fmt = FuncFormatter(lambda v, _: f"{int(v):,}")
d = df.copy()
d["날짜"] = d["날짜"].apply(lambda s: datetime.strptime(s, "%Y%m%d"))

# ---- 그래프 1: 가장 최근 하루의 TOP10 (당일관객수) ----
last = d["날짜"].max()
day = d[d["날짜"] == last].sort_values("순위")

fig, ax = plt.subplots(figsize=(9, 5.5))
bars = ax.barh(day["영화제목"], day["당일관객수"], color="#2F6DB5", height=0.6)
ax.invert_yaxis()                                    # 1위가 맨 위로
for b, v in zip(bars, day["당일관객수"]):
    ax.text(v, b.get_y() + b.get_height() / 2, f"  {v:,}", va="center", fontsize=10)
ax.set_title(f"{last:%Y-%m-%d} 일일 박스오피스 TOP10 (당일관객수)", loc="left", fontsize=14)
ax.xaxis.set_major_formatter(fmt)
ax.set_xlabel("당일관객수(명)")
ax.set_xlim(0, day["당일관객수"].max() * 1.18)
for s in ("top", "right"): ax.spines[s].set_visible(False)
ax.grid(axis="x", alpha=0.25)
plt.tight_layout()
plt.savefig("boxoffice_day_top10.png", dpi=150)
plt.show()

# ---- 그래프 2: 10일간 관객수 추이 (10일 합계 상위 5편) ----
top5 = d.groupby("영화제목")["당일관객수"].sum().nlargest(5).index
pv = d[d["영화제목"].isin(top5)].pivot_table(index="날짜", columns="영화제목", values="
당일관객수")
pv = pv[list(top5)]                                   # 합계 큰 순서로 고정
colors = ["#0072B2", "#D55E00", "#009E73", "#CC79A7", "#E69F00"]
markers = ["o", "s", "^", "D", "v"]

fig, ax = plt.subplots(figsize=(10, 5.5))
for name, c, m in zip(pv.columns, colors, markers):
    ax.plot(pv.index, pv[name], color=c, marker=m, markersize=6, linewidth=2,
label=name)
ax.set_title("최근 10일 일일 관객수 추이 (10일 합계 상위 5편)", loc="left", fontsize=14)
ax.yaxis.set_major_formatter(fmt)
ax.set_ylabel("당일관객수(명)")
ax.set_xticks(pv.index)
ax.set_xticklabels([t.strftime("%m/%d") for t in pv.index])
ax.legend(frameon=False, loc="upper left", bbox_to_anchor=(1.01, 1))
for s in ("top", "right"): ax.spines[s].set_visible(False)
ax.grid(axis="y", alpha=0.25)
plt.tight_layout()
```

```
plt.savefig("boxoffice_trend.png", dpi=150)
plt.show()
```

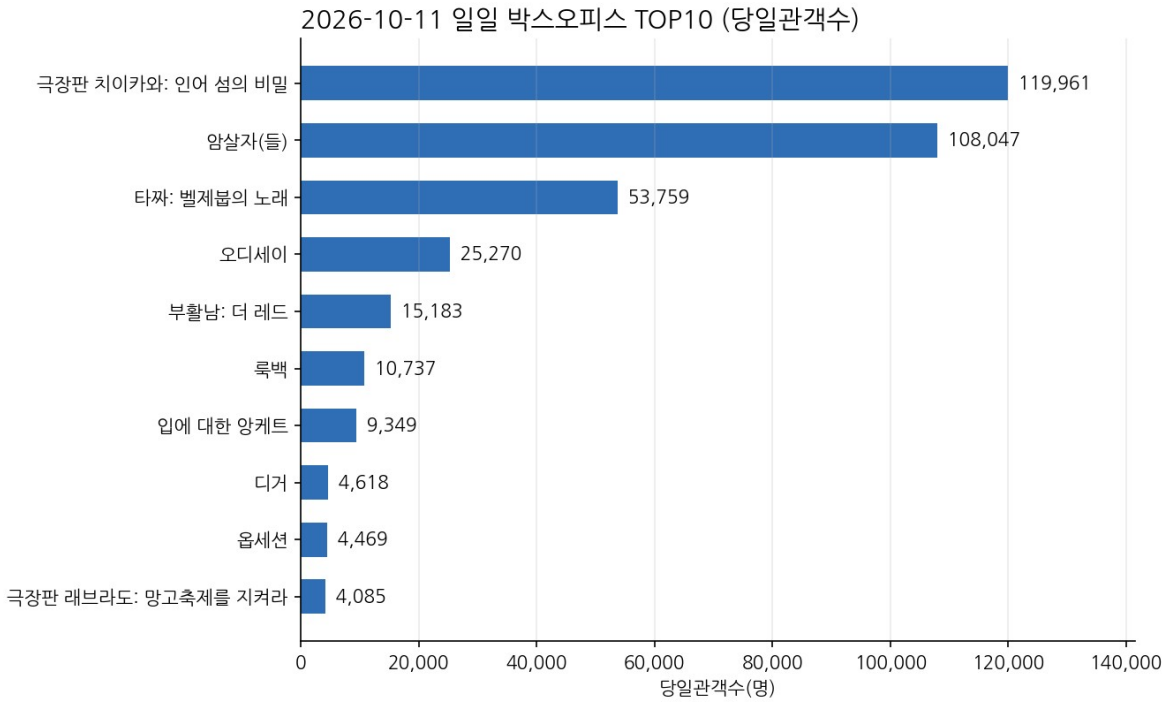


그림 C-1. 가장 최근 하루의 일일 박스오피스 TOP10 — 순위는 위에서 아래로, 막대 끝의 숫자는 당일관객수

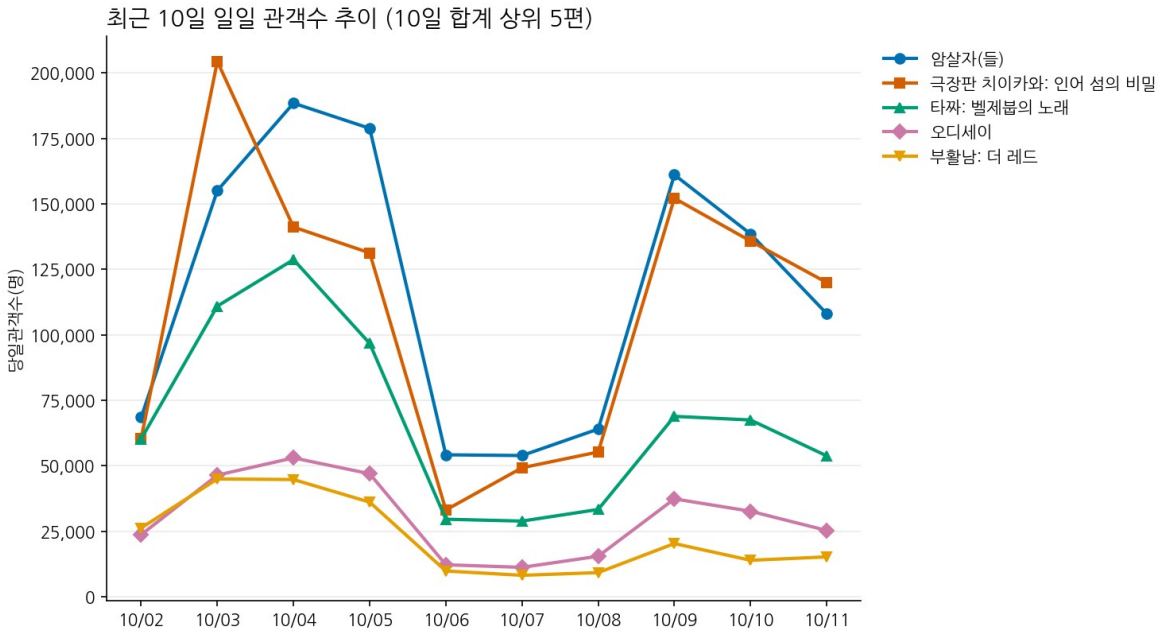


그림 C-2. 최근 10 일 일일 관객수 추이 — 10 일 합계 상위 5 편. 선이 끊긴 구간은 그날 TOP10 밖이었다는 뜻

그래프 읽는 법

- 그림 C-1 에서 1 위는 119,961 명, 2 위는 108,047 명이고 3 위부터는 격차가 크게 벌어집니다. 순위는 한 줄 숫자보다 막대로 보면 격차가 바로 보입니다.

- 그림 C-2 에서 모든 영화가 같은 날 함께 오르내립니다. 10/6~10/8 에는 모두 크게 줄고, 10/3~10/5 와 10/9 이후에 다시 늘어납니다. 영화의 인기보다 **요일과 휴일의 영향**이 먼저 보입니다.
- 1 위는 대부분 날짜에 암살자(들)이지만 10/3 과 10/11 에는 극장판 치이카와가 1 위입니다. 선이 교차하는 지점이 순위가 바뀐 날입니다.
- 그래프는 무슨 일이 있었는지만 보여 줍니다. 왜 그랬는지(연휴 때문인지, 개봉 효과인지)는 데이터에 없으므로 기사에 쓰기 전에 따로 확인합니다.

셀 3 — 10 일 관객수 합계 계산하기 (셀 1 을 실행한 뒤 이어서 실행합니다. df 를 그대로 씁니다.)

> **목표**
> 지난 10 일(2026-10-02 ~ 10-11) 동안 영화별 관객수 합계를 구하고, 합계가 큰 순서로 정렬합니다. 그리고 그 값이 맞는지 다른 방법으로 한 번 더 확인합니다. 아래 코드가 답입니다.

```
# 1) 영화별 10 일 합계 (당일관객수를 더한다)
total = (df.groupby("영화제목")
           .agg(TOP10 에든날=("날짜", "nunique"),
                관객수합계=("당일관객수", "sum"),
                매출액합계=("당일매출액", "sum"))
           .sort_values("관객수합계", ascending=False)
           .reset_index())
total.insert(0, "순위", range(1, len(total) + 1))

# 2) 검증: 누적관객수의 증가분으로 다시 계산해 본다
chk = (df.sort_values("날짜")
         .groupby("영화제목")
         .agg(첫날누적=("누적관객수", "first"),
              끝날누적=("누적관객수", "last"),
              첫날당일=("당일관객수", "first")))
chk["누적증가분"] = chk["끝날누적"] - chk["첫날누적"] + chk["첫날당일"]
total = total.merge(chk[["누적증가분"]], left_on="영화제목", right_index=True)
total["일치"] = total["관객수합계"] == total["누적증가분"]

total.to_csv("boxoffice_last10days_total.csv", index=False, encoding="utf-8-sig")
display(total[["순위", "영화제목", "TOP10 에든날", "관객수합계", "누적증가분", "
일치"]].head(12))
```

실행 결과 — 10 일 동안 한 번이라도 TOP10 에 든 영화는 16 편이고, 아래는 합계 상위 12 편입니다. 결과는 boxoffice_last10days_total.csv 로 저장됩니다.

순위	영화제목	TOP10 에 든 날	관객수 합계	누적 증가분	일치
1	암살자(들)	10	1,170,596	1,170,596	True
2	극장판 치이카와: 인어 섬의 비밀	10	1,082,699	1,082,699	True
3	타짜: 벨제붑의 노래	10	678,427	678,427	True
4	오디세이	10	304,020	304,020	True
5	부활남: 더 레드	10	228,448	228,448	True

순위	영화제목	TOP10 에 든 날	관객수 합계	누적 증가분	일치
6	옵세션	10	72,284	72,284	True
7	디거	9	68,600	68,600	True
8	룩백	4	61,979	61,979	True
9	가능한 사랑	3	60,533	60,533	True
10	인턴	7	58,988	58,988	True
11	입에 대한 앙케트	5	50,474	50,474	True
12	어벤져스: 엔드게임 앙코르	6	44,253	46,933	False

결과 읽는 법

- 1 위 암살자(들)의 10 일 합계는 1,170,596 명, 2 위 극장판 치이카와는 1,082,699 명으로 두 영화의 차이는 약 8 만 8 천 명입니다. 이 순위는 앞의 그림 C-2 의 선 모양과도 맞습니다.
- **일치 열은 두 가지 방법으로 구한 값이 같은지를 비교한 결과입니다. 방법 1 은 TOP10 에 든 날의 당일관객수를 모두 더한 관객수합계, 방법 2 는 누적관객수로 다시 계산한 누적증가분(끝날 누적 − 첫날 누적 + 첫날 당일)입니다.**
- **True 이면 두 값이 같다는 뜻이고, 합계가 두 방법으로 확인되었다는 신호입니다.**
- **False 이면 두 값이 다르다는 뜻이므로 이유를 찾아야 합니다. 방법 1 은 TOP10 에 든 날만 더할 수 있고, 방법 2 의 누적관객수에는 TOP10 밖이었던 날의 관객수도 들어 있기 때문입니다. 어벤져스: 엔드게임 앙코르는 합계 44,253 명, 증가분 46,933 명으로 달랐습니다. 저장된 CSV 전체(16 편)에서는 극장판 래브라도(12,086 명 대 21,720 명)와 레지던트 이블(4,343 명 대 14,162 명)도 False 입니다. 세 영화 모두 TOP10 에 든 날이 중간에 끊겨 있습니다. 기사에 쓸 때 True 는 합계를 그대로 써도 되고, False 는 "TOP10 에 든 날 기준 합계"라고 밝히거나 누적 증가분을 사용합니다.**
- 이 합계는 **TOP10 에 든 날의 관객수만 더한 값**입니다. 기사나 보고서에 쓸 때는 “KOBIS 일일 박스오피스 TOP10 집계 기준”이라고 밝히고, **TOP10 에 든 날이 10 일인 영화**만 10 일 전체 합계로 소개합니다.

저장할 때는 4 원칙의 ‘기록’대로 출처(영화진흥위원회 KOBIS)와 수집일시를 함께 남깁니다. 위 코드는 수집일시 열을 포함하고, 출처는 보고서나 기사에 표기합니다.

수집한 데이터의 출처 · 수집일시 · 조건:

C-4. 왜 계산은 파이썬으로 해야 하나

> **이 절의 결론**
> 기사에 들어가는 숫자는 파이썬이 계산합니다. AI 는 코드를 만들어 주고 결과를 문장으로 풀어 쓰는 일을 맡고, 최종 확인은 사람이 합니다. 숫자 계산을 AI 챗봇에 직접 시키지 않습니다.

1. 파이썬의 계산에는 할루시네이션이 없다

파이썬은 코드에 적힌 연산을 규칙대로 실행합니다. 1,170,596 이라는 합계를 그럴듯하게 "예측" 하는 것이 아니라 100 행의 숫자를 실제로 더합니다. 그래서 행을 빼먹거나, 숫자를 바꿔 쓰거나, 없는 합계를 지어내는 일이 없습니다. 같은 데이터에 같은 코드를 실행하면 누가 몇 번을 실행해도 같은 값이 나옵니다.

생성형 AI 는 다음에 올 글자를 예측해 문장을 만드는 방식이어서, CSV 를 붙여넣고 합계를 시키면 계산이 아니라 계산처럼 보이는 글을 쓸 수 있습니다. 같은 질문을 다시 하면 값이 달라지기도 하고, 행이 많을수록 위험이 커집니다. 코드 실행 기능이 있는 AI 는 내부에서 파이썬으로 계산하므로 숫자는 믿을 수 있지만, 그때도 계산을 한 것은 AI 가 아니라 파이썬입니다.

2. 틀릴 수 있는 것은 계산이 아니라 코드에 적은 지시

파이썬은 계산을 틀리지 않지만, 시킨 내용이 잘못되면 잘못된 지시를 정확하게 계산합니다. 예를 들어 당일관객수 대신 누적관객수를 더하면 틀린 합계가 매번 똑같이 나옵니다. 이번 실습의 False 도 계산 오류가 아닙니다. 코드가 "TOP10 에 든 날만 더하라" 는 범위로 계산했기 때문에 나온 차이입니다.

그래서 파이썬을 쓸 때의 순서는 정해져 있습니다. 올바른 열을 더하도록 코드를 쓰고, 셀 3 처럼 다른 방법으로 한 번 더 계산해 비교하고, 기사에는 집계 기준(TOP10 에 든 날 기준)을 밝힙니다. 계산은 파이썬이, 지시가 맞는지의 확인은 사람이 합니다.

3. 파이썬을 쓰면 좋은 점

- **정확합니다.** 사람이 100 행을 눈으로 더하면 한 자리만 틀려도 기사가 틀립니다. 파이썬은 10 만 행이어도 같은 코드로 같은 정확도로 계산합니다.
- **다시 실행할 수 있습니다.** 코드 맨 위의 DAYS = 10 을 30 으로 바꾸면 30 일 집계가 됩니다. 날짜가 바뀌어도 같은 코드를 다시 실행하면 됩니다.
- **과정이 코드로 남습니다.** 어떤 열을 어떻게 더했는지 코드를 보면 알 수 있으므로 다른 사람이 검증할 수 있습니다. 팩트체크와 출처 표기에도 필요한 근거가 됩니다.
- **계산한 표가 바로 그래프가 됩니다.** 같은 표(df)를 그림 C-1, C-2 처럼 몇 줄의 코드로 시각화할 수 있고, 숫자만 봐서는 안 보이는 패턴이 드러납니다. 이번 실습에서는 연휴 전후로 모든 영화가 함께 오르내리는 것이 그래프에서 먼저 보였습니다.
- **AI 와 함께 쓸 수 있습니다.** 코드를 직접 외울 필요는 없습니다. 하고 싶은 계산을 AI 에게 말해 코드를 만들게 하고, 그 코드를 Colab 에서 실행하면 계산은 파이썬이 합니다. 결과 숫자는 이 교재처럼 다른 방법으로 확인합니다.

4. 시각화에 쓰는 대표 라이브러리

라이브러리	잘하는 것	이 실습에서의 예
matplotlib	막대·선·산점도 같은 기본 그래프를 세밀하게 조정	그림 C-1(막대), 그림 C-2(선) — 이 교재에서 실제로 사용

라이브러리	잘하는 것	이 실습에서의 예
seaborn	matplotlib 위에서 통계용 그래프를 짧은 코드로, 보기 좋은 기본 스타일로 그림	영화 × 날짜별 당일관객수를 색의 진하기로 보는 히트맵(sns.heatmap)
plotly	마우스를 올리면 값이 보이고 확대·축소되는 인터랙티브 그래프, HTML 로 저장해 공유	10 일 추이를 영화별 선으로 보는 인터랙티브 그래프(px.line)

5. 숫자·문장·확인, 누가 맡나

일	맡길 도구
숫자 계산 (합계·평균·순위)	파이썬(pandas)
코드 작성과 결과 해석, 기사 문장 초안	AI
숫자와 문장의 최종 확인	사람 — 3 원칙 ③ 사람의 검수

6. 파이썬과 AI 챗봇 직접 계산 비교

항목	파이썬(pandas)	AI 챗봇에 CSV 를 붙여넣고 계산
계산 방식	코드에 적은 연산을 규칙대로 실행	다음에 올 글자를 예측해 숫자를 씀 (코드 실행 기능이 있으면 내부에서 파이썬 사용)
같은 질문을 다시 하면	항상 같은 결과	값이 달라질 수 있음
행이 많아지면	처리 시간만 늘어남	행 누락·숫자 오류 위험이 커짐
과정 확인	코드와 검증 셀로 확인 가능	어떻게 계산했는지 알기 어려움
틀렸을 때	지시(코드)를 고치면 해결	원인을 찾기 어렵고 재현도 어려움

직접 비교해 보고 싶다면 같은 CSV(boxoffice_last10days.csv)를 AI 챗봇에 붙여넣고 “지난 10 일 영화별 관객수 합계 TOP5 를 알려줘” 라고 요청한 뒤, 셀 3 의 결과와 한 줄씩 대조합니다. 값이 같더라도 같은 요청을 한 번 더 했을 때 같은 값이 나오는지 함께 확인합니다.

Part D. 시트에서 자동으로 수집하기 (Apps Script)

Part C 의 Colab 은 **사람이 셀을 실행해야** 데이터가 모입니다. 구글시트에 내장된 **Apps Script** 는 같은 API 호출을 시트 안에서 하고, **정해진 시간에 자동으로 실행**할 수 있습니다. 우리는 Part C 에서 쓴 KOBIS 일일 박스오피스를 그대로 이어서, 매일 아침 시트에 순위가 자동으로 쌓이게 만듭니다.

D-1. 세 가지 방법 비교

방법	특징	언제 쓰나
IMPORTXML (Part A)	수식 하나로 웹페이지에서 가져오기. 코드 없음	공개된 정적 페이지, 한 번 확인할 때

방법	특징	언제 쓰나
Colab · 키 없이 (Part B)	파이썬 코드로 RSS·표·목록 페이지 수집, 정리·CSV 저장까지 한 흐름. 실행은 사람이 함	공개된 페이지를 반복·대량 수집하고 표로 정리할 때
Colab · API (Part C)	파이썬 코드. 가공·시각화에 강함. 실행은 사람이 함	분석·그래프, 대량 반복 호출
Apps Script (Part D)	시트에 내장된 자바스크립트. 시간 트리거로 자동 실행. 무료	매일·매시간 같은 데이터를 계속 모을 때

Colab 자체에는 무료 예약 실행 기능이 없어서, 자동 수집은 시트 쪽에서 겁니다.

D-2. 함수 두 개의 역할

함수	역할	실행 횟수
collectBoxOffice	KOBIS API 를 호출하고 결과를 시트에 한 줄씩 기록하는 '작업' 함수	자동 실행될 때마다 (처음엔 직접 테스트)
setupTrigger	"collectBoxOffice 를 매일 오전 8 시에 실행해줘" 라고 예약만 걸어 두는 함수. 알람 시계를 맞추는 역할	딱 한 번

D-3. 준비 — 구글시트와 Apps Script 열기

1. 새 구글시트를 만든다. 이름은 "박스오피스 자동수집"처럼 알아보기 쉽게 짓는다.
2. 메뉴에서 **확장 프로그램 → Apps Script** 를 눌러 편집기를 연다.
3. 기본으로 들어 있는 코드(function myFunction() {})를 **모두 지운다**.
4. 아래 D-4 의 코드를 통째로 붙여넣고 저장(Ctrl+S)한다.

> **키는 코드에 쓰지 않습니다**
> KOBIS 인증키는 코드가 아니라 **스크립트 속성**에 저장합니다. 편집기 왼쪽 ⚙️ **프로젝트 설정** → 아래쪽 **스크립트 속성** → **스크립트 속성 추가**에서 속성 이름은 KOBIS_KEY, 값은 내가 발급받은 키를 넣고 저장합니다. 이렇게 하면 코드를 공유해도 키가 새지 않습니다.

D-4. 코드 — 어제의 박스오피스를 시트에 기록

```
const SHEET_NAME = "박스오피스로그";

// 작업 함수: KOBIS에서 어제 일일 박스오피스를 받아 시트에 추가한다
function collectBoxOffice() {
  const key = PropertiesService.getScriptProperties().getProperty("KOBIS_KEY");
  if (!key) { throw new Error("스크립트 속성에 KOBIS_KEY를 먼저 등록하세요."); }

  const now = new Date();
  const yesterday = new Date(now.getTime() - 24 * 60 * 60 * 1000);
  const target = Utilities.formatDate(yesterday, "Asia/Seoul", "yyyyMMdd");
  const targetText = Utilities.formatDate(yesterday, "Asia/Seoul", "yyyy-MM-dd");

  const sheet = getOrCreateSheet_();
  // 같은 날짜를 두 번 기록하지 않도록 확인
  const lastRow = sheet.getLastRow();
  if (lastRow > 1) {
    const dates = sheet.getRange(2, 2, lastRow - 1, 1).getDisplayValues().flat();
    if (dates.indexOf(targetText) !== -1) {
      Logger.log(targetText + " 은(는) 이미 기록되어 있어 건너뜁니다.");
      return;
    }
  }

  const url =
"http://kobis.or.kr/kobisopenapi/webservice/rest/boxoffice/searchDailyBoxOfficeList.json"
            + "?key=" + key + "&targetDt=" + target;
  const res = UrlFetchApp.fetch(url, { muteHttpExceptions: true });
  const data = JSON.parse(res.getContentText());
  const list = data.boxOfficeResult && data.boxOfficeResult.dailyBoxOfficeList;
  if (!list) {
    Logger.log("오류: " + res.getContentText().substring(0, 300));
    return;
  }

  const collectedAt = Utilities.formatDate(now, "Asia/Seoul", "yyyy-MM-dd HH:mm");
  const rows = list.map(function (m) {
    return [collectedAt, targetText, Number(m.rank), m.movieNm,
            Number(m.audiCnt), Number(m.audiAcc), Number(m.salesAmt)];
  });
  sheet.getRange(sheet.getLastRow() + 1, 1, rows.length,
rows[0].length).setValues(rows);
  Logger.log(rows.length + "행 기록 완료 (" + targetText + ")");
}
```

이어서 같은 편집기 창에 아래 코드를 **바로 아래에 이어 붙입니다.** (시트 만들기 + 자동 실행 예약)

```
// 시트가 없으면 만들고 제목 줄을 넣는다
function getOrCreateSheet_() {
  const ss = SpreadsheetApp.getActiveSpreadsheet();
  let sheet = ss.getSheetByName(SHEET_NAME);
  if (!sheet) {
    sheet = ss.insertSheet(SHEET_NAME);
    sheet.appendRow(["수집일시", "조회일", "순위", "영화제목", "당일관객수", "누적관객수", "
당일매출액"]);
    sheet.getRange("B:B").setNumberFormat("@");   // 조회일을 날짜로 바꾸지 않고 글자로 둔다
  }
  return sheet;
}

// 예약 함수: 매일 오전 8시에 collectBoxOffice를 실행하도록 등록 (딱 한 번만 실행)
function setupTrigger() {
  ScriptApp.getProjectTriggers().forEach(function (t) {
    if (t.getHandlerFunction() === "collectBoxOffice") { ScriptApp.deleteTrigger(t); }
  });
  ScriptApp.newTrigger("collectBoxOffice")
    .timeBased().everyDays(1).atHour(8).inTimezone("Asia/Seoul").create();
  Logger.log("트리거 등록 완료: 매일 오전 8시 자동 실행");
}
```

D-5. 실행 순서

1. **저장합니다.** 편집기 위쪽 제목 옆에 “저장되지 않은 변경사항”이 보이면 아직 저장 전입니다. 이 상태에서는 ▷ 실행과 디버그 버튼이 흐리고, 함수 선택 칸에는 “함수 없음”이라고 나옵니다. 도구줄의 디스크 모양 저장 아이콘(또는 Ctrl+S)을 누르면 문구가 사라지고 함수를 고를 수 있게 됩니다.
2. **KOBIS_KEY 가 등록되어 있는지 확인합니다.** 왼쪽 ⚙️ 프로젝트 설정 → 스크립트 속성에 KOBIS_KEY 가 있어야 합니다(위 D-3 의 박스). 없으면 실행했을 때 “KOBIS_KEY 를 먼저 등록하세요” 오류가 납니다. 확인한 뒤 왼쪽 <> 편집기 아이콘을 눌러 코드 화면으로 돌아옵니다.
3. **collectBoxOffice 를 실행합니다. (시험 수집)** 도구줄의 함수 선택 칸에서 collectBoxOffice 를 고르고 ▷ 실행을 누릅니다. 목록에는 collectBoxOffice 와 setupTrigger 두 개만 보이는 것이 정상입니다. getOrCreateSheet_는 이름 끝에 _가 붙은 보조 함수라서 목록에 나오지 않습니다. 처음 실행할 때는 권한 검토 창이 뜹니다. 내 구글 계정을 고르고 “확인되지 않은 앱”이라고 나오면 고급 → 이동 → 허용을 누릅니다.
4. **실행 로그를 확인합니다.** 화면 아래 실행 로그에 “10 행 기록 완료 (어제 날짜)”와 “실행이 완료됨”이 나오면 성공입니다. 오류가 나오면 D-6 을 봅니다.
5. **스프레드시트 탭으로 돌아갑니다.** Apps Script 편집기가 아니라 처음에 만든 구글시트 탭으로 돌아가 아래 시트 탭 목록에 박스오피스로그가 생겼는지, 10 행이 들어 있는지 확인합니다. (안 보이면 새로고침 F5)
6. **setupTrigger 를 실행합니다. (예약 등록, 딱 한 번)** 다시 Apps Script 편집기로 가서 함수 선택 칸을 setupTrigger 로 바꾸고 ▷ 실행을 누릅니다. 권한 창이 또 뜨면 같은 방법으로 허용합니다. 실행 로그에 “트리거 등록 완료: 매일 오전 8 시 자동 실행”이 나오면 성공입니다. (그림 D-1)
7. **트리거가 등록되었는지 확인합니다.** 트리거는 “정해진 시간에 이 함수를 자동으로 실행해 줘”라고 걸어 두는 예약입니다. 왼쪽 메뉴의 ⏰ 알람 시계 모양 아이콘(트리거)을 누르면 등록된 예약이 표로 나옵니다.

위쪽에 “트리거 1개 표시 중”이라고 나오고, 함수가 collectBoxOffice 이고 이벤트가 “시간 기반”인 줄이 한 줄 있으면 끝입니다. (그림 D-2) 최종 실행과 오류율이 “-”인 것은 아직 자동 실행이 한 번도 일어나지 않았다는 뜻으로 정상입니다. 내일 아침 8 시에서 9 시 사이 이후에는 최종 실행에 실행 시각이 나타납니다.



그림 D-1. setupTrigger 를 실행한 뒤의 실행 로그 — “트리거 등록 완료”가 나오면 예약이 만들어진 것

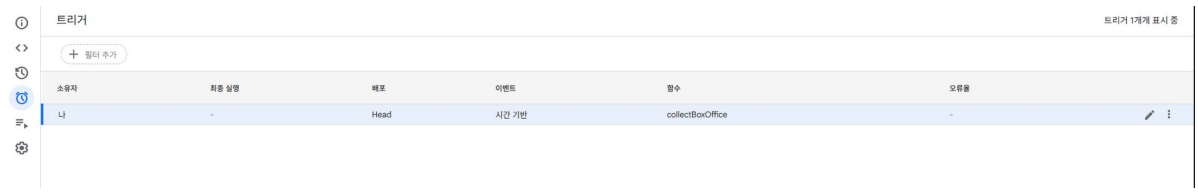


그림 D-2. 왼쪽 ⏰ 트리거 메뉴 — collectBoxOffice 가 “시간 기반”으로 등록된 모습

함수 두 개를 언제 실행하나

함수	하는 일	실행 횟수
collectBoxOffice	어제의 박스오피스 TOP10 을 시트에 기록	시험으로 직접 한 번 실행하고, 이후에는 매일 오전 8 시에 자동으로 실행됨
setupTrigger	collectBoxOffice 를 매일 오전 8 시에 실행하도록 예약	딱 한 번. 다시 실행해도 기존 예약을 지우고 새로 만들어 중복되지 않음

이후에는 아무것도 하지 않아도 매일 아침 전날의 박스오피스 TOP10 이 시트에 쌓입니다. 같은 날짜는 두 번 기록되지 않도록 막아 두었습니다.

A. 수집일시	B. 조회일	C. 순위	D. 영화제목	E. 당일관객수	F. 누적…
2026-10-11 08:00	2026-10-10	1	(영화 제목)	(숫자)	…

로그에 나온 문구와 시트에 생긴 행 수:

D-6. 막혔을 때

증상	점검
“KOBIS_KEY 를 먼저 등록하세요” 오류	⚙️ 프로젝트 설정 → 스크립트 속성에 이름을 KOBIS_KEY(대문자·밑줄 정확히)로 등록했는지 확인
로그에 “오류: …”가 나옴	인증키 값을 다시 확인한다. 키를 막 발급받았다면 승인·반영까지 시간이 걸릴 수 있다
시트 탭이 안 보임	편집기 화면이 아니라 원래 스프레드시트 탭에서 확인하고 새로고침(F5)
“이미 기록되어 있어 건너뜁니다”	정상. 어제 날짜가 이미 있다는 뜻이다. 다시 시험하려면 시트의 해당 행을 지운다
권한 승인 창에서 “확인되지 않은 앱”	내가 만든 스크립트이므로 고급 → 이동 → 허용
트리거를 여러 번 만들었나 걱정될 때	setupTrigger 는 기존 트리거를 먼저 지우고 새로 만들어 중복되지 않는다

> **Part C 와 비교해 보세요**
> Colab 에서는 requests.get(url, params)로 호출하고 CSV 로 저장했습니다. Apps Script 에서는 UrlFetchApp.fetch(url)로 호출하고 시트에 바로 기록합니다. 호출 주소와 JSON 구조(boxOfficeResult → dailyBoxOfficeList)는 **똑같습니다.**

코드 읽기 연습

- Colab 코드의 TARGET_DATE 와 Apps Script 의 target 은 어떻게 같고 다른가요?
- “같은 날짜를 두 번 기록하지 않는” 부분은 어떤 줄이 맡고 있나요?
- 실행 시각을 오전 8 시가 아니라 9 시로 바꾸려면 어느 줄을 고쳐야 하나요?

내 답과 메모:

> **확장 아이디어**
> 며칠 쌓인 시트는 Part C-3 의 10 일 집계처럼 영화별로 더해 TOP10 을 만들 수 있습니다. 매일 자동 수집이 곧 ‘기간 집계’의 재료가 됩니다.

Part E. 저작권 확인 후 AI 로 정제하기

이번 주에는 챗봇을 새로 만들지 않습니다. **2 주차에 만든 내 도메인 챗봇**에 수집한 데이터를 넣어 콘텐츠로 정제합니다.

E-1. 저작권 유형 먼저 확인

수집한 사이트가 언론사인지 공공기관인지, 공공기관이라면 공공누리 몇 유형인지 사이트 하단·이용약관에서 확인합니다. 소재는 3 주차 폴더의 「3 주차_뉴스브랜드트렌드_데이터출처목록」에서 고릅니다. 이 목록에는 AI 정제 가능 여부가 없으므로, 고른 출처는 아래 표와 E-4 의 소재 표로 확인합니다.

출처	AI 정제	방법
공공누리 1 유형	가능	출처를 표시하고 정제. 영리 서비스에도 쓸 수 있음
공공누리 2 유형	수업만 가능	출처를 표시하고 정제하되, 상업적 이용 금지라서 영리 서비스에는 쓸 수 없음
공공누리 3·4 유형 (변경금지)	제한	변경금지라서 사람이 먼저 요약 → AI 는 표현만 다듬기. 영리 서비스에는 쓰지 않음
언론사 기사 · 포털 뉴스	제한	헤드라인 1 개 + 출처 표기까지만. 본문은 사람이 읽고 재구성
기업 보도자료	주의	텍스트 인용은 비교적 안전, 이미지·작성자 표기는 별도 확인

내 수집 출처는? ☐ 언론사 (저작권 있음) ☐ 공공기관 — 공공누리 (☐ 1 유형 ☐ 2 유형 ☐ 3 유형 ☐ 4 유형)

> **'제한' 에 해당하면**
> AI 에게 원문을 통째로 넣지 마세요. **내가 읽고 요약한 글**을 근거로 삼고, AI 는 그 글의 표현만 다듬습니다.

E-2. 정제 흐름

① 수집	② 복사	③ 챗봇 입력	④ 프롬프트	⑤ 결과	⑥ 검수
구글시트로 표 만들기	F 열까지 채운 표를 복사	내 도메인 챗봇에 붙여넣기	역할 · 구조 · 톤 지정	콘텐츠 확인	원문과 대조

프롬프트는 2 주차 '도메인별 프롬프트 가이드' 의 템플릿을 그대로 재사용합니다.

내 도메인: ☐ 뉴스 ☐ 브랜드 ☐ 트렌드 사용한 챗봇: ☐ Gemini Gems ☐ Claude Projects

E-3. 방법 A — Gem 으로 정제하기 (표 형식, 카드뉴스용)

정제 규칙은 Gem 의 지시문에 한 번만 저장해 두고, 매번 6 열 표만 올립니다. 결과는 카드뉴스 템플릿에 옮겨 적습니다. 연습 소재는 E-1 에서 “가능”으로 확인한 자료(예: 한국소비자원 피해예방주의보)로 합니다.

Gem 만들기

1. **Gem 을 만듭니다.** gemini.google.com → 사이드바의 Gems → 새 Gem 을 열고 이름을 “카드뉴스 정제봇”으로 짓습니다. 화면 순서는 2 주차 「LLM 챗봇 만들기 안내자료」 4 장과 같습니다.
2. **지시문을 붙여넣습니다.** 아래 박스 전체를 ‘요청 사항’ 칸에 붙여넣습니다. 도메인이 뉴스가 아니면 2 주차 도메인별 템플릿으로 규칙만 바꿉니다.
3. **미리보기에서 시험하고 저장합니다.** 오른쪽 미리보기에 6 열 표를 올려 결과를 확인한 뒤 저장합니다.

```
당신은 뉴스 헤드라인을 카드뉴스용 문구로 다듬는 편집자입니다.
사용자가 올리는 스프레드시트(열: 구분, 분야, URL, 헤드라인, 기사링크, 기사원문)를 읽고,
'기사원문' 열의 본문만을 근거로 각 행을 아래 규칙에 따라 정제해 표로 출력합니다.

[정제 규칙]
1. 헤드라인: 원본 제목을 공백 포함 15 자 이내로 줄여 다시 쓴다. 설명이 아니라 핵심 사실 하나를 앞에 둔다. 원
제목과 본문에 없는 단어를 새로 만들지 않는다.
2. 강조 수치: 본문에 나온 구체적 숫자 중 가장 핵심적인 것 하나를 쓰고, 괄호 안에 그 수치가 무엇을 잰 값인지
짧게 적는다. 예: 1,898 가구(임시주택 거주 가구). 숫자가 없으면 "자료 없음"이라고 쓴다.
3. 요약: 1~2 문장으로 핵심만 담는다.
   - 본문에 없는 사실, 수치, 발언을 추가하지 않는다.
   - 숫자와 날짜는 본문 그대로 쓰고 어림하거나 바꿔 쓰지 않는다. 강조 수치와 요약의 숫자는 같아야 한다.
   - 정부·군·기관 같은 주체, 의혹·혐의 같은 정도 표현을 서로 바꾸지 않는다.
   - 누가 밝히고 발표하고 보도한 내용인지 문장에 쓴다. 본문이 "~가 밝혔다", "~ 매체가 보도했다"라고 쓴
내용은 사실처럼 단정하지 않는다.
4. 본문 문장을 그대로 옮기지 말고 새로운 표현으로 바꿔 쓴다. 단, 규칙 1~3 을 어기지 않는 범위에서 바꾼다.
5. 기사원문 칸이 비어 있는 행은 정제하지 말고 "기사원문 없음"이라고 표시한다.
6. 본문에 날짜가 없으면 날짜 표현을 쓰지 않는다. 서로 다른 기사의 사실을 합쳐 새로운 인과관계를 만들지
않는다.
7. 카테고리는 시트의 '구분'과 '분야' 값을 그대로 쓴다. 예: 정치 · 국방. 새 분류를 만들지 않는다.
8. 행이 4 개보다 많으면 카드뉴스에 쓸 만한 4 개만 고른다. 헤드라인에 구체적 수치가 있는 사실 전달형 기사를
우선하고, 고른 기준을 한국어로 한 줄만 쓴다.

[작업 방식]
9. 파일을 만들거나 코드를 실행하지 말고 바로 텍스트로 답한다. 기사 본문을 화면에 그대로 출력하지 않는다.
10. 출력 전에 각 행의 헤드라인, 수치, 요약을 본문과 한 번 대조하고, 어긋나는 부분을 고친 뒤에 출력한다.

[출력 형식]
표로만 출력한다: 번호 | 카테고리 | 헤드라인(정제됨) | 강조 수치 | 요약(정제됨)
표 아래에 선정 기준을 한 줄로 쓴다. 그 외 설명은 쓰지 않는다.
```

표를 준비하고 요청하기

1. **열 이름을 맞춥니다.** 시트 1 행의 열 이름을 구분, 분야, URL, 헤드라인, 기사링크, 기사원문으로 둡니다. 열 이름이 다르면 지시문과 어긋나므로 시트에서 고칩니다.
2. **값으로 바꿉니다.** D~F 열은 수식이므로 전체를 복사해 Ctrl+Shift+V(값만 붙여넣기)로 붙여넣은 뒤 올립니다. 수식 상태로 올리면 수식 문자열이 읽힐 수 있습니다.
3. **새 대화에서 요청합니다.** 표(CSV/Excel)를 올리고 “첨부한 표를 규칙대로 정제해줘.” 한 줄만 입력합니다. 규칙은 지시문에 이미 들어 있으므로 다시 쓰지 않습니다.

4. **Claude Projects 로 할 때는** Instructions 칸에 같은 지시문을 넣고, 표는 대화에 올립니다. 규칙과 제한은 Gem 과 같습니다.

내가 바꾼 부분(도메인에 맞게 수정한 프롬프트):

E-4. 정제 프롬프트 — 요령 · 규칙 · 제한 (Gems · Claude 공통)

Gemini Gems 와 Claude Projects 는 지시문을 저장해 두고 쓰는 방식이 같아서, 아래 요령과 규칙과 제한이 똑같이 적용됩니다. 지시문을 넣는 칸만 다릅니다. (Gems: '요청 사항', Claude Projects: Instructions)

1. 프롬프트 요령

- **고정과 변동을 나눕니다.** 역할, 규칙, 출력 형식은 지시문에 한 번 저장하고, 이번에 쓸 표와 소재는 채팅창에 올립니다.
- **지시문은 순서대로 씁니다.** 역할 → 근거로 쓸 열 → 번호 붙은 규칙 → 출력 형식 순입니다.
- **근거 범위를 못 박습니다.** "기사원문 열의 본문만을 근거로" 처럼 어디서만 가져올지 지정합니다.
- **규칙은 하나에 하나씩, 구체적으로 씁니다.** "정확하게 쓴다" 보다 "정부와 군을 서로 바꾸지 않는다" 처럼 씁니다.
- **출력 형식을 고정합니다.** 표로만 출력하고 그 외 설명을 쓰지 않게 합니다.
- **처음에는 소량만 넣습니다.** 행이 많을수록 실수가 늘어납니다. 지시문이 안정된 뒤에 늘립니다.
- **고쳤으면 새 대화에서 다시 시험합니다.** 이미 열려 있던 대화에는 바뀐 지시문이 반영되지 않을 수 있습니다.
- **같은 표로 두 번 돌려 비교합니다.** 두 결과에서 값이 다른 부분이 AI 가 불안정한 곳이므로 먼저 확인합니다.

2. 지시문에 꼭 넣을 규칙

규칙	지시문에 쓰는 문장	막으려는 오류
근거 밖 사실 금지	본문에 없는 사실, 수치, 발언을 추가하지 않는다.	지어낸 내용
숫자 그대로	숫자와 날짜는 본문 그대로 쓰고 어림하거나 바꿔 쓰지 않는다.	4 일을 사흘로, 1 천 898 가구를 약 1 천 900 가구로
주체·정도 유지	정부·군·기관 같은 주체, 의혹·혐의 같은 정도 표현을 서로 바꾸지 않는다.	정부를 군으로, 의혹을 혐의로
출처 귀속	누가 밝히고 발표하고 보도한 내용인지 문장에 쓴다.	발표나 보도를 사실처럼 단정
새 단어 금지	원 제목과 본문에 없는 단어를 새로 만들지 않는다.	뜻이 안 통하는 헤드라인 단어
수치의 의미	강조 수치 옆 괄호에 무엇을 잰 값인지 적는다.	무엇의 수치인지 빠짐
날짜·혼합 금지	날짜가 없으면 날짜 표현을 쓰지 않는다. 다른 기사의 사실을 합치지 않는다.	없는 날짜, 엉뚱한 인과관계

규칙	지시문에 쓰는 문장	막으려는 오류
코드·본문 출력 금지	파일을 만들거나 코드를 실행하지 말고 바로 텍스트로 답한다. 본문을 화면에 그대로 출력하지 않는다.	코드 실행, 기사 본문 전체 출력
출력 전 자가 대조	출력 전에 각 행을 본문과 한 번 대조하고 고친 뒤 출력한다.	위 오류의 일부

3. 제한 — 규칙을 넣어도 남는 것

- **오류는 줄일 뿐 없애지 못합니다.** 이 교재의 시험에서 지시문을 고친 뒤에도 일부 오류가 반복되었습니다. AI 결과는 초안입니다.
- **같은 요청도 결과가 달라질 수 있습니다.** 한 번 맞았다고 다음에도 맞는다는 보장이 없습니다.
- **Gem 이 지시와 달리 코드를 실행할 수 있습니다.** 표를 파일로 올리면 읽으려고 코드를 실행하고 본문을 화면에 쏟아낸 경우가 있었습니다. 표를 복사해 채팅창에 붙여넣는 방법을 시험해 볼 수 있습니다.
- **숫자는 코드로, 주체와 표현은 사람이 확인합니다.** 결과의 숫자가 원문에 있는지는 파이썬으로 대조할 수 있고, 주어나 표현 변화는 사람이 읽어야 잡힙니다. 마지막 확인은 E-6 검수에서 합니다.

시험에서 실제로 나온 오류 유형

유형	원문	AI 결과
숫자 바꿈	4 일 만에	사흘 만에
숫자 어림	1 천 898 가구	약 1 천 900 가구
주체 바꿈	정부가 공개	우리 군의 ~ 공개
정도 바꿈	위반 의혹	위반 혐의
없는 단어	현장조사 불복	현장조사 훔친
출처 누락	매체가 보도했다 · 합참이 밝혔다	사실처럼 단정

4. 소재 제한 — 저작권 유형

수업 실습과 영리 서비스는 기준이 다릅니다. 공공누리 유형은 한국저작권위원회 안내(1 유형: 상업·변경 가능, 2 유형: 상업 금지, 3 유형: 변경 금지, 4 유형: 둘 다 금지)를 기준으로 했습니다.

소재	수업 실습	영리 서비스
공공누리 1 유형	가능	가능 (출처 표시)
공공누리 2 유형	가능	불가 (상업적 이용 금지)
공공누리 3 유형	제한	제한 (변경금지 — AI 재구성은 변경에 해당할 수 있음)
공공누리 4 유형	제한	불가

소재	수업 실습	영리 서비스
언론사 기사 · 포털 뉴스 본문	제한 (헤드라인 + 출처까지)	언론사와 이용 계약·협의 필요 (한국신문협회는 동의 없는 AI 이용을 저작권 침해로 본다고 밝힘)
네이버 뉴스	제한	제한 (본문은 언론사 저작물. 검색 API 는 제목·링크·짧은 발췌문만 주고, 개정 약관이 AI 요약·학습과 자동 수집을 막는다는 보도가 있음 — 약관 직접 확인)
기업 보도자료	주의	주의 (이미지·작성자 표기는 별도 확인)
“AI 학습 및 활용 금지” 가 적힌 자료	제한	제한

이 표는 법률 자문이 아닙니다. 공공누리 유형은 게시물마다 다를 수 있으므로 사이트 하단 표시를 직접 확인하고, 영리 서비스를 시작하기 전에는 전문가에게 이용 조건을 확인합니다.

E-5. 방법 B — Gemini Canvas 로 카드뉴스 슬라이드 자동 생성

1. Gemini 에서 내 Gem(또는 새 대화)을 열고 도구 메뉴에서 **Canvas** 를 켠다.
2. 수집한 스프레드시트(CSV/Excel)를 업로드한다.
3. 아래 프롬프트를 붙여넣어 실행하면 Canvas 에 슬라이드 세트가 만들어진다.
4. 마음에 안 드는 부분은 대화하듯 수정을 요청한다. (예: “헤드라인 폰트 더 크게”)
5. 완성되면 **구글 슬라이드로 내보내기**를 눌러 파일로 저장한다.

```
당신은 뉴스·정보를 카드뉴스 프레젠테이션으로 만드는 디자이너 겸 편집자입니다.
업로드된 스프레드시트(열: 구분, 분야, URL, 헤드라인, 기사링크, 기사원문)를 읽고,
‘기사원문’ 열의 실제 본문을 근거로 카드뉴스 프레젠테이션을 만들어주세요.

규칙:
1. 슬라이드마다: 카테고리 태그, 임팩트 있는 헤드라인(15 자 내외로 재구성),
   핵심 수치 강조, 1~2 문장 요약(원문에 없는 내용 추가 금지), 출처 표시를 포함한다.
2. 원문 문장을 그대로 베끼지 말고 반드시 새로운 표현으로 바꿔 쓴다.
3. 행이 4 개보다 많으면 카드뉴스에 쓸 만한 4 개만 골라 정리한다.
4. 첫 슬라이드는 표지(제목만), 마지막 슬라이드는 마무리로 구성한다.
5. 디자인 톤: 남색과 빨강 포인트 컬러, 깔끔한 흰 배경.

전체 프레젠테이션으로 만들어주세요.
```

AI 가 형식을 어기거나 내용을 지어낸다면

프롬프트 끝에 한 줄을 추가합니다: “반드시 지정된 형식으로만 답하고, 기사원문에 없는 사실은 절대 추가하지 마라.” 구글 슬라이드는 기본 16:9 이므로, 정사각형(1:1) 카드뉴스로 쓰려면 내보낸 뒤 [파일] → [페이지 설정]에서 크기를 바꿉니다.

E-6. 검수 — 사람이 마지막으로 확인

AI 가 만든 결과를 원문(F 열 기사원문 또는 원자료)과 대조하고 체크합니다.

☐ 모든 수치가 원자료와 일치한다

☐ 출처(기관명·발표일)를 표시했다

☐ 원문에 없는 사실이나 추측 표현이 들어가지 않았다

☐ 수치가 무엇을 측정한 것인지(신고·접수 건수인지, 실제 전체 규모인지) 구분해 표현했다

☐ 원문 문장을 그대로 옮기지 않았다

AI 가 만든 정제 결과(붙여넣기):

검수 결과 — 고친 곳과 이유:

이번 주 정리

- 수집(Part A~D)과 정제(Part E)를 직접 연결해 보니, 어느 단계가 가장 시간이 오래 걸렸나요?
- 저작권 확인 단계는 왜 필요했나요?
- 같은 목적이라면 구글시트(IMPORTXML), Colab, Apps Script 중 무엇을 고르겠습니까? 이유는?
- 매일 자동으로 쌓이는 데이터는 어떤 기사·콘텐츠에 쓸 수 있을까요?

나의 정리: